

Chapter 16

Reusing Classes

In Chapter ??, we developed classes to implement Conway’s Game of Life. We can reuse the `Cell` and `GridCanvas` classes to implement other simulations. One of the most interesting zero-player games is *Langton’s Ant*, which models an “ant” that walks around a grid. The ant follows only two simple rules:

1. If the ant is on a white cell, it turns to the right, makes the cell black, and moves forward.
2. If the ant is on a black cell, it turns to the left, makes the cell white, and moves forward.

Because the rules are simple, you might expect the ant to do something simple, like make a square or repeat a simple pattern. But starting on a grid with all white cells, the ant makes more than 10,000 steps in a seemingly random pattern before it settles into a repeating loop of 104 steps. You can read more about it at [https://en.wikipedia.org/wiki/Langton’s_ant](https://en.wikipedia.org/wiki/Langton's_ant).

In this chapter, we present an implementation of Langton’s Ant and use it to demonstrate more advanced object-oriented techniques.

16.1 Langton’s Ant

We begin by defining a `Langton` class that has a grid and information about the ant. The constructor takes the grid dimensions as parameters:

```
public class Langton {
    private GridCanvas grid;
    private int xpos;
    private int ypos;
    private int head; // 0=North, 1=East, 2=South, 3=West

    public Langton(int rows, int cols) {
        grid = new GridCanvas(rows, cols, 10);
        xpos = rows / 2;
        ypos = cols / 2;
        head = 0;
    }
}
```

`grid` is a `GridCanvas` object, which represents the state of the cells. `xpos` and `ypos` are the coordinates of the ant, and `head` is the “heading” of the ant; that is, which direction it is facing. `head` is an integer with four possible values, where 0 means the ant is facing “north” (i.e., toward the top of the screen), 1 means “east”, etc.

Here’s an `update` method that implements the rules for Langton’s Ant:

```
public void update() {
    flipCell();
    moveAnt();
}
```

The `flipCell` method gets the `Cell` at the ant’s location, figures out which way to turn, and changes the state of the cell:

```
private void flipCell() {
    Cell cell = grid.getCell(xpos, ypos);
    if (cell.isOff()) {
        head = (head + 1) % 4;    // turn right
        cell.turnOn();
    } else {
        head = (head + 3) % 4;    // turn left
        cell.turnOff();
    }
}
```

We use the remainder operator, %, to make `head` wrap around: if `head` is 3 and we turn right, it wraps around to 0; if `head` is 0 and we turn left, it wraps around to 3.

Notice that to turn right, we add 1 to `head`. To turn left, we could subtract 1, but `-1 % 4` is `-1` in Java. So we add 3 instead, since one left turn is the same as three right turns.

The `moveAnt` method moves the ant forward one square, using `head` to determine which way is forward:

```
private void moveAnt() {
    if (head == 0) {
        ypos -= 1;
    } else if (head == 1) {
        xpos += 1;
    } else if (head == 2) {
        ypos += 1;
    } else {
        xpos -= 1;
    }
}
```

Here is the main method we use to create and display the `Langton` object:

```
public static void main(String[] args) {
    String title = "Langton's Ant";
    Langton game = new Langton(61, 61);
    JFrame frame = new JFrame(title);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setResizable(false);
    frame.add(game.grid);
    frame.pack();
    frame.setVisible(true);
    game.mainloop();
}
```

Most of this code is the same as the `main` we used to create and run `Conway`, in Section ???. It creates and configures a `JFrame` and runs `mainloop`.

And that’s everything! If you run this code with a grid size of 61 x 61 or larger, you will see the ant eventually settle into a repeating pattern.

Because we designed `Cell` and `GridCanvas` to be reusable, we didn’t have to modify them at all. However, we now have two copies of `main` and `mainloop`—one in `Conway`, and one in `Langton`.

16.2 Refactoring

Whenever you see repeated code like `main`, you should think about ways to remove it. In Chapter ??, we used inheritance to eliminate repeated code. We’ll do something similar with `Conway` and `Langton`.

First, we define a superclass named `Automaton`, in which we will put the code that `Conway` and `Langton` have in common:

```
public class Automaton {
    private GridCanvas grid;

    public void run(String title, int rate) {
        JFrame frame = new JFrame(title);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setResizable(false);
        frame.add(this.grid);
        frame.pack();
        frame.setVisible(true);
        this.mainloop(rate);
    }
}
```

`Automaton` declares `grid` as an instance variable, so every `Automaton` “has a” `GridCanvas`. It also provides `run`, which contains the code that creates and configures the `JFrame`.

The `run` method takes two parameters: the window `title` and the frame `rate`; that is, the number of time steps to show per second. It uses `title` when creating the `JFrame`, and it passes `rate` to `mainloop`:

```
private void mainloop(int rate) {
    while (true) {

        // update the drawing
        this.update();
        grid.repaint();

        // delay the simulation
        try {
            Thread.sleep(1000 / rate);
        } catch (InterruptedException e) {
            // do nothing
        }
    }
}
```

`mainloop` contains the code you first saw in Section ?? . It runs a `while` loop forever (or until the window closes). Each time through the loop, it runs `update` to update `grid` and then `repaint` to redraw the grid.

Then it calls `Thread.sleep` with a delay that depends on `rate`. For example, if `rate` is 2, we should draw two frames per second, so the delay is a half second, or 500 milliseconds.

This process of reorganizing existing code, without changing its behavior, is known as **refactoring**. We're almost finished; we just need to redesign `Conway` and `Langton` to extend `Automaton`.

16.3 Abstract Classes

If we were not planning to implement any other zero-person games, we could leave well enough alone. But there are a few problems with the current design:

- The `grid` attribute is `private`, making it inaccessible in `Conway` and `Langton`. We could make it `public`, but then other (unrelated) classes would have access to it as well.
- The `Automaton` class has no constructors, and even if it did, there would be no reason to create an instance of this class.

- The `Automaton` class does not provide an implementation of `update`. In order to work properly, subclasses need to provide one.

Java provides language features to solve these problems:

- We can make the `grid` attribute `protected`, which means it's accessible to subclasses but not other classes.
- We can make the class `abstract`, which means it cannot be instantiated. If you attempt to create an object for an abstract class, you will get a compiler error.
- We can declare `update` as an `abstract` method, meaning that it must be overridden in subclasses. If the subclass does not override an abstract method, you will get a compiler error.

Here's what `Automaton` looks like as an abstract class (using the methods `mainloop` and `run` from Section ??):

```
public abstract class Automaton {
    protected GridCanvas grid;

    public abstract void update();

    private void mainloop(int rate) {
        // this method invokes update
    }

    public void run(String title, int rate) {
        // this method invokes mainloop
    }
}
```

Notice that the `update` method has no body. The declaration specifies the name, arguments, and return type. But it does not provide an implementation, because it is an abstract method.

Notice also the word `abstract` on the first line, which declares that `Automaton` is an abstract class. In order to have any abstract methods, a class must be declared as abstract.

Any class that extends `Automaton` must provide an implementation of `update`; the declaration here allows the compiler to check.

Here's what `Conway` looks like as a subclass of `Automaton`:

```
public class Conway extends Automaton {  
  
    // same methods as before, except mainloop is removed  
  
    public static void main(String[] args) {  
        String title = "Conway's Game of Life";  
        Conway game = new Conway();  
        game.run(title, 2);  
    }  
}
```

`Conway` extends `Automaton`, so it inherits the `protected` instance variable `grid` and the methods `mainloop` and `run`. But because `Automaton` is abstract, `Conway` has to provide `update` and a constructor (which it has already).

Abstract classes are essentially “incomplete” class definitions that specify methods to be implemented by subclasses. But they also provide attributes and methods to be inherited, thus eliminating repeated code.

16.4 UML Diagram

At the beginning of the chapter, we had three classes: `Cell`, `GridCanvas`, and `Conway`. We then developed `Langton`, which had almost the same `main` and `mainloop` methods as `Conway`. So we refactored the code and created `Automaton`. Figure ?? summarizes the final design.

The diagram shows three examples of inheritance: `Conway` is an `Automaton`, `Langton` is an `Automaton`, and `GridCanvas` is a `Canvas`. It also shows two examples of composition: `Automaton` has a `GridCanvas`, and `GridCanvas` has a 2D array of `Cells`.

The diagram also shows that `Automaton` uses `JFrame`, `GridCanvas` uses `Graphics`, and `Cell` uses `Graphics` and `Color`.

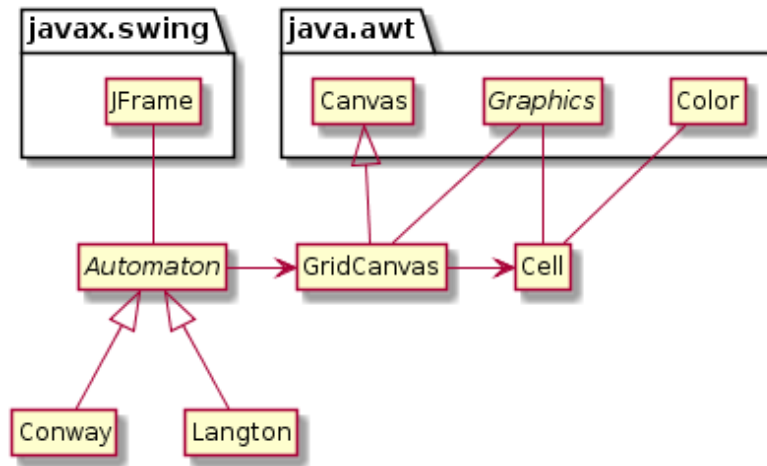


Figure 16.1: UML class diagram of Conway and Langton applications.

Automaton is in italics to indicate that it is an abstract class. As it happens, *Graphics* is an abstract class, too.

Conway and Langton are **concrete classes**, because they provide an implementation for all of their methods. In particular, they implement the `update` method that was declared **abstract** in *Automaton*.

One of the challenges of object-oriented programming is keeping track of a large number of classes and the relationships between them. UML class diagrams can help.

16.5 Vocabulary

refactor: To restructure or reorganize existing source code without changing its behavior.

abstract class: A class that is declared as **abstract**; it cannot be instantiated, and it may (or may not) include abstract methods.

concrete class: A class that is *not* declared as **abstract**; each of its methods must have an implementation.

16.6 Exercises

The code for this chapter is in the *ch16* directory of *ThinkJavaCode2*. See page x for instructions on how to download the repository. Before you start the exercises, we recommend that you compile and run the examples.

Exercise 16.1 The last section of this chapter introduced `Automaton` as an abstract class and rewrote `Conway` as a subclass of `Automaton`. Now it's your turn: rewrite `Langton` as a subclass of `Automaton`, removing the code that's no longer needed.

Exercise 16.2 Mathematically speaking, Game of Life and Langton's Ant are *cellular automata*. "Cellular" means it has cells, and "automaton" means it runs itself. See https://en.wikipedia.org/wiki/Cellular_automaton for more discussion.

Implement another cellular automaton of your choice. You may have to modify `Cell` and/or `GridCanvas`, in addition to extending `Automaton`. For example, Brian's Brain (https://en.wikipedia.org/wiki/Brian's_Brain) requires three states: "on", "dying", and "off".

Chapter 17

Advanced Topics

When we first looked at inheritance in Chapter ??, our purpose was to avoid duplicating code. We noticed that “decks of cards” and “hands of cards” had common functionality, and we designed a `CardCollection` class to provide it. This technique is an example of **generalization**. By generalizing the code, we were able to reuse it in the `Deck` and `Hand` classes.

In Chapter ??, we looked at inheritance from a different point of view. When designing `GridCanvas` to represent a grid of cells, we extended `Canvas` and overrode its `paint` method. This design is an example of **specialization**. Using the code provided by `Canvas`, we created a specialized subclass with minimal additional code.

We didn’t write the code for `Canvas`; it’s part of the Java library. But we were able to customize it for our own purposes. In fact, the `Canvas` class was explicitly designed to be extended.

In this chapter, we’ll explore the concept of inheritance more fully and present event-driven programming. We’ll continue to develop graphical simulations as a running example, but this time in varying shapes and colors!

17.1 Polygon Objects

The word polygon means “many angles”; the most basic polygons are triangles (three angles), rectangles (four angles), pentagons (five angles), and so forth.

Polygons are an important part of computer graphics because they are used to compose more complex images.

Java provides a `Polygon` class (in `java.awt`) that we can use to represent and draw polygons. The following code creates an empty `Polygon` and adds three points, forming a triangle:

```
Polygon p = new Polygon();  
p.addPoint(57, 110);  
p.addPoint(100, 35);  
p.addPoint(143, 110);
```

Internally, `Polygon` objects have three attributes:

- `public int npoints;` `// total number of points`
- `public int[] xpoints;` `// array of X coordinates`
- `public int[] ypoints;` `// array of Y coordinates`

When a `Polygon` is created, `npoints` is 0 and the two arrays are initialized with length 4. As points are added, `npoints` is incremented. If `npoints` exceeds the length of the arrays, larger arrays are created, and the previous values are copied over (similar to how `ArrayList` works).

The `Polygon` class provides many useful methods, like `contains`, `intersects`, and `translate`. We'll get to those later, but first we're going to do some specialization.

17.2 Adding Color

Specialization is useful for adding new features to an existing class, especially when you can't (or don't want to) change its design. For example, we can extend the `Polygon` class by adding a `draw` method and a `Color` attribute:

```
public class DrawablePolygon extends Polygon {
    protected Color color;

    public DrawablePolygon() {
        super();
        color = Color.GRAY;
    }

    public void draw(Graphics g) {
        g.setColor(color);
        g.fillPolygon(this);
    }
}
```

As a reminder, constructors are not inherited when you extend a class. If you don't define a constructor, the compiler will generate one that does nothing.

The constructor for `DrawablePolygon` uses `super` to invoke the constructor for `Polygon`, which initializes the attributes `npoints`, `xpoints`, and `ypoints`. Then `DrawablePolygon` initializes the `color` attribute to `GRAY`.

`DrawablePolygon` has the same attributes and methods that `Polygon` has, so you can use `addPoint` as before, or you can directly access `npoints`, `xpoints`, and `ypoints` (since they are `public`). You can also use methods like `contains`, `intersects`, and `translate`.

The following code creates a `DrawablePolygon` with the same points as in the previous section and sets its color to `GREEN`:

```
DrawablePolygon p = new DrawablePolygon();
p.addPoint(57, 110);
p.addPoint(100, 35);
p.addPoint(143, 110);
p.color = Color.GREEN;
```

17.3 Regular Polygons

In mathematics, a regular polygon has all sides the same length and all angles equal in measure. Regular polygons are a special case of polygons, so we will use specialization to define a class for them.

We could extend the `Polygon` class, as we did in the previous section. But then we would not have the `Color` functionality we just added. So we will make `RegularPolygon` extend `DrawablePolygon`.

To construct a `RegularPolygon`, we specify the number of sides, the radius (distance from the center to a vertex), and the color. For example:

```
RegularPolygon rp = new RegularPolygon(6, 50, Color.BLUE);
```

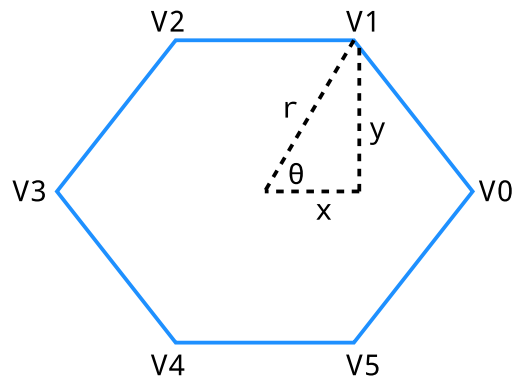


Figure 17.1: Determining the x and y coordinates of vertex $V1$, given the radius r and angle θ . The center of the polygon is at the origin $(0, 0)$.

The constructor uses trigonometry to find the coordinates of each vertex. Figure ?? illustrates the process. The number of sides ($n = 6$) and the radius ($r = 50$) are given as parameters.

- Imagine a clock hand starting at $V0$ and rotating counterclockwise to $V1$, $V2$, and so forth. In Figure ??, the hand is currently at $V1$.
- The angle θ is $2\pi/n$, since there are 2π radians in a circle. In other words, we are dividing the rotation of the clock hand into n equal angles.
- By definition, $\cos(\theta) = x/r$ and $\sin(\theta) = y/r$. Therefore, $x = r \cos(\theta)$ and $y = r \sin(\theta)$.
- We can determine the other (x, y) coordinates by multiplying θ by i , where i is the vertex number.

Here is the constructor for `RegularPolygon`:

```
public RegularPolygon(int nsides, int radius, Color color) {  
  
    // initialize DrawablePolygon attributes  
    this.npoints = nsides;  
    this.xpoints = new int[nsides];  
    this.ypoints = new int[nsides];  
    this.color = color;  
  
    // the amount to rotate for each vertex (in radians)  
    double theta = 2.0 * Math.PI / nsides;  
  
    // compute x and y coordinates, centered at the origin  
    for (int i = 0; i < nsides; i++) {  
        double x = radius * Math.cos(i * theta);  
        double y = radius * Math.sin(i * theta);  
        xpoints[i] = (int) Math.round(x);  
        ypoints[i] = (int) Math.round(y);  
    }  
}
```

This constructor initializes all four `DrawablePolygon` attributes, so it doesn't have to invoke `super()`.

It initializes `xpoints` and `ypoints` by creating arrays of integer coordinates. Inside the `for` loop, it uses `Math.sin` and `Math.cos` (see Section 4.6) to compute the coordinates of the vertices as floating-point numbers. Then it rounds them off to integers and stores them in the arrays.

When we construct a `RegularPolygon`, the vertices are centered at the point $(0, 0)$. If we want the center of the polygon to be somewhere else, we can use `translate`, which we inherit from `Polygon`:

```
RegularPolygon rp = new RegularPolygon(6, 50, Color.BLUE);  
rp.translate(100, 100);
```

The result is a six-sided polygon with radius 50 centered at the point $(100, 100)$.

17.4 More Constructors

Classes in the Java library often have more than one constructor for convenience. We can do the same with `RegularPolygon`. For example, we can make the `color` parameter optional by defining a second constructor:

```
public RegularPolygon(int nsides, int radius) {  
    this(nsides, radius, Color.GRAY);  
}
```

The keyword `this`, when used in a constructor, invokes another constructor in the same class. It has a similar syntax as the keyword `super`, which invokes a constructor in the superclass.

Similarly, we could make the `radius` parameter optional too:

```
public RegularPolygon(int nsides) {  
    this(nsides, 50);  
}
```

Now, suppose we invoke the `RegularPolygon` constructor like this:

```
RegularPolygon rp = new RegularPolygon(6);
```

Because we provide only one integer argument, Java calls the third constructor, which calls the second one, which calls the first one. The result is a `RegularPolygon` with the specified value of `nsides`, 6, the default value of `radius`, 50, and the default color, `GRAY`.

When writing constructors, it's a good idea to validate the values you get as arguments. Doing so prevents run-time errors later in the program, which makes the code easier to debug.

For `RegularPolygon`, the number of sides should be at least three, the radius should be greater than zero, and the color should not be `null`. We can add the following lines to the first constructor:


```
public RegularPolygon(int nsides, int radius, Color color) {  
  
    // validate the arguments  
    if (nsides < 3) {  
        throw new IllegalArgumentException("invalid nsides");  
    }  
    if (radius <= 0) {  
        throw new IllegalArgumentException("invalid radius");  
    }  
    if (color == null) {  
        throw new NullPointerException("invalid color");  
    }  
  
    // the rest of the method is omitted  
}
```

In this example, we `throw` an exception to indicate that one of the arguments is invalid. By default, these exceptions terminate the program and display an error message along with the stack trace.

Because we added this code to the most general constructor, we don't have to add it to the others.

17.5 An Initial Drawing

Now that we have `DrawablePolygon` and `RegularPolygon`, let's take them for a test drive. We'll need a `Canvas` for drawing them, so we define a new class, `Drawing`, that extends `Canvas`:

```
public class Drawing extends Canvas {  
    private ArrayList<DrawablePolygon> list;  
  
    public Drawing(int width, int height) {  
        setSize(width, height);  
        setBackground(Color.WHITE);  
        list = new ArrayList<DrawablePolygon>();  
    }  
}
```

```
public void add(DrawablePolygon dp) {
    list.add(dp);
}

public void paint(Graphics g) {
    for (DrawablePolygon dp : list) {
        dp.draw(g);
    }
}
```

The `Drawing` class has an `ArrayList` of `DrawablePolygon` objects. When we create a `Drawing` object, the list is initially empty. The `add` method takes a `DrawablePolygon` and adds it to the list.

`Drawing` overrides the `paint` method that it inherits from `Canvas`. `paint` loops through the list of `DrawablePolygon` objects and invokes `draw` on each one.

Here is an example that creates three `RegularPolygon` objects and draws them. Figure ?? shows the result.

```
public static void main(String[] args) {

    // create some regular polygons
    DrawablePolygon p1 = new RegularPolygon(3, 50, Color.GREEN);
    DrawablePolygon p2 = new RegularPolygon(6, 50, Color.ORANGE);
    DrawablePolygon p3 = new RegularPolygon(360, 50, Color.BLUE);

    // move them out of the corner
    p1.translate(100, 80);
    p2.translate(250, 120);
    p3.translate(400, 160);

    // create drawing, add polygons
    Drawing drawing = new Drawing(500, 250);
    drawing.add(p1);
    drawing.add(p2);
    drawing.add(p3);
}
```

```
// set up the window frame
JFrame frame = new JFrame("Drawing");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.add(drawing);
frame.pack();
frame.setVisible(true);
}
```

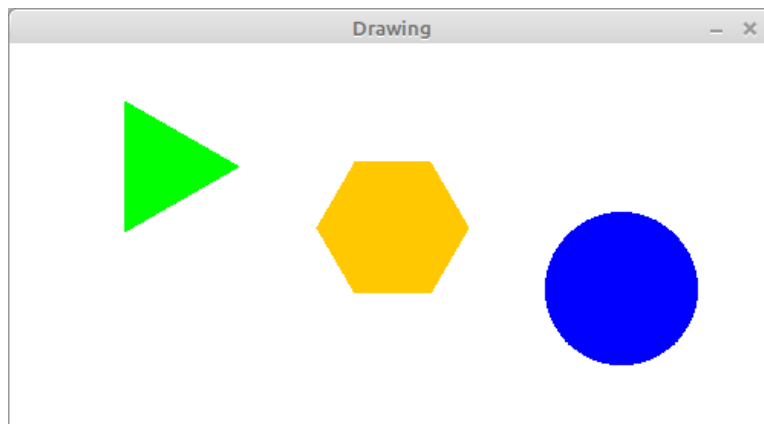


Figure 17.2: Initial drawing of three `RegularPolygon` objects.

The first block of code creates `RegularPolygon` objects with 3, 6, and 360 sides. As you can see, a polygon with 360 sides is a pretty good approximation of a circle.

The second block of code translates the polygons to different locations. The third block of code creates the `Drawing` and adds the polygons to it. And the fourth block of code creates a `JFrame`, adds the `Drawing` to it, and displays the result.

Most of these pieces should be familiar, but one part of this program might surprise you. When we create the `RegularPolygon` objects, we assign them to `DrawablePolygon` variables. It might not be obvious why that's legal.

`RegularPolygon` extends `DrawablePolygon`, so every `RegularPolygon` object is also a `DrawablePolygon`. The parameter of `Drawing.add` has to be a `DrawablePolygon`, but it can be any type of `DrawablePolygon`, including `RegularPolygon` and other subclasses.

This design is an example of **polymorphism**, a fancy word that means “having many forms”. `Drawing.add` is a polymorphic method, because the parameter can be one of many types. And the `ArrayList` in `Drawing` is a polymorphic data structure, because the elements can be different types.

17.6 Blinking Polygons

At this point, we have a simple program that draws polygons; we can make it more fun by adding animation. Chapter ?? introduced the idea of simulating time steps. Here’s a loop that runs the animation:

```
while (true) {
    drawing.step();
    try {
        Thread.sleep(1000 / 30);
    } catch (InterruptedException e) {
        // do nothing
    }
}
```

Each time through the loop, we call `step` to update the `Drawing`. Then we sleep with a delay calculated to update about 30 times per second.

Here’s what the `step` method of `Drawing` looks like:

```
public void step() {
    for (DrawablePolygon dp : list) {
        dp.step();
    }
    repaint();
}
```

It invokes `step` on each `DrawablePolygon` in the list and then repaints (clears and redraws) the canvas.

In order for this code to compile, we need `DrawablePolygon` to provide a `step` method. Here’s a version that doesn’t do anything; we’ll override it in subclasses:

```
public void step() {  
    // do nothing  
}
```

Now let's design a new type of polygon that blinks. We'll define a class named `BlinkingPolygon` that extends `RegularPolygon` and adds two more attributes: `visible`, which indicates whether the polygon is visible, and `count`, which counts the number of time steps since the last blink:

```
public class BlinkingPolygon extends RegularPolygon {  
    protected boolean visible;  
    protected int count;  
  
    public BlinkingPolygon(int nsides, int radius, Color c) {  
        super(nsides, radius, c);  
        visible = true;  
        count = 0;  
    }  
  
    public void draw(Graphics g) {  
        if (visible) {  
            super.draw(g);  
        }  
    }  
  
    public void step() {  
        count++;  
        if (count == 10) {  
            visible = !visible;  
            count = 0;  
        }  
    }  
}
```

The constructor uses `super` to call the `RegularPolygon` constructor. Then it initializes `visible` and `count`. Initially, the `BlinkingPolygon` is visible.

The `draw` method draws the polygon only if it is visible. It uses `super` to call `draw` in the parent class. But the parent class is `RegularPolygon`, which

does not provide a `draw` method. In this case, `super` invokes `draw` from the `DrawablePolygon` class.

The `step` method increments `count`. Every 10 time steps, it toggles `visible` and resets `count` to 0.

17.7 Interfaces

You might be getting tired of polygons at this point. Can't we draw anything else? Of course we can, but `Drawing` is currently based on `DrawablePolygon`. To draw other types of objects, we have to generalize the code.

The `Drawing` class does essentially three things: (1) it maintains a list of objects, (2) it invokes the `draw` method on each object, and (3) it invokes the `step` method on each object.

So here's one way we could make the code more general:

1. Define a new superclass, which we call `Actor`, that provides the two methods needed by `Drawing`:

```
public class Actor {
    public void draw(Graphics g) {
        // do nothing
    }
    public void step() {
        // do nothing
    }
}
```

2. In the `Drawing` class, replace `DrawablePolygon` with `Actor`.
3. Any class that we want to draw must now extend `Actor`.

There's just one problem: `DrawablePolygon` already extends `Polygon`, and classes can extend only one superclass. Also, the `Actor` class seems pointless, since the methods it defines don't do anything.

Java provides another mechanism for inheritance that solves these problems. We can define `Actor` as an `interface` instead of a `class`, like this:

```
public interface Actor {  
    void draw(Graphics g);  
    void step();  
}
```

Like a class definition, an **interface** definition contains methods. But it contains only the declarations of the methods, not their implementations.

Like an abstract class, an interface specifies methods that must be provided by subclasses. The difference is that an abstract class can implement some methods; an interface cannot.

All interface methods are **public** by default, since they are intended to be used by other classes. So there is no need to declare them as **public**.

To inherit from an interface, you use the keyword **implements** instead of **extends**. Here's a version of `DrawablePolygon` that extends `Polygon` and implements `Actor`. So it inherits methods from `Polygon`, and it is required to provide the methods in `Actor`; namely `draw` and `step`:

```
public class DrawablePolygon extends Polygon implements Actor {  
    // rest of the class omitted  
}
```

In terms of inheritance, `DrawablePolygon` is both a `Polygon` and an `Actor`. So the following assignments are legal:

```
Polygon p1 = new DrawablePolygon();  
Actor a2 = new DrawablePolygon();
```

And the same is true for subclasses of `DrawablePolygon`; these assignments are legal too:

```
Polygon p2 = new RegularPolygon(5, 50, Color.YELLOW);  
Actor a2 = new RegularPolygon(5, 50, Color.YELLOW);
```

Interfaces are another example of polymorphism. `a1` and `a2` are the same type of variable, but they refer to objects with different types. And similarly with `p1` and `p2`.

Classes may extend only one superclass, but they may implement as many interfaces as needed. Java library classes often implement multiple interfaces.

17.8 Event Listeners

Now that our `Drawing` is based on `Actor` instead of `DrawablePolygon`, we can draw other types of graphics. Here is the beginning of a class that reads an image from a file and shows the image moving across the canvas. The class is called `Sprite` because a moving image is sometimes called a **sprite**, in the context of computer graphics:

```
public class Sprite implements Actor, KeyListener {
    private int xpos;
    private int ypos;
    private int dx;
    private int dy;
    private Image image;

    public Sprite(String path, int xpos, int ypos) {
        this.xpos = xpos;
        this.ypos = ypos;
        try {
            this.image = ImageIO.read(new File(path));
        } catch (IOException exc) {
            exc.printStackTrace();
        }
    }
}
```

The instance variables `xpos` and `ypos` represent the location of the sprite. `dx` and `dy` represent the velocity of the sprite in the x and y directions.

The constructor takes as parameters the name of a file and the initial position. It uses `ImageIO`, from the `javax.imageio` package, to read the file. If an error occurs during reading, an `IOException` is caught, and the program displays the stack trace for debugging.

`Sprite` implements two interfaces: `Actor` and `KeyListener`. `Actor` requires that we provide `draw` and `step` methods:


```
public void draw(Graphics g) {  
    g.drawImage(image, xpos, ypos, null);  
}  
  
public void step() {  
    xpos += dx;  
    ypos += dy;  
}
```

The `draw` method draws the image at the sprite's current position. The `step` method changes the position based on `dx` and `dy`, which are initially zero.

`KeyListener` is an interface for receiving keyboard events, which means we can detect and respond to key presses. A class that implements `KeyListener` has to provide the following methods:

```
void keyPressed(KeyEvent e)
```

Invoked when a key has been “pressed”. This method is invoked repeatedly while a key is being held down.

```
void keyReleased(KeyEvent e)
```

Invoked when a key has been “released”, meaning it is no longer down.

```
void keyTyped(KeyEvent e)
```

Invoked when a key has been “typed”, which generally means it has been both pressed and released.

These methods get invoked when the user presses and releases *any* key. They take a `KeyEvent` object as a parameter, which specifies which key was pressed, released, or typed.

We can use these methods to design a simple animation using the arrow keys. When the user presses up or down, the sprite will move up or down. When the user presses left or right, the sprite will move left or right.

Here's an implementation of `keyPressed` that uses a `switch` statement to test which arrow key was pressed and sets `dx` or `dy` accordingly. (There is no `default` branch, so we ignore all other keys.)

```
public void keyPressed(KeyEvent e) {  
    switch (e.getKeyCode()) {  
        case KeyEvent.VK_UP:  
            dy = -5;  
            break;  
        case KeyEvent.VK_DOWN:  
            dy = +5;  
            break;  
        case KeyEvent.VK_LEFT:  
            dx = -5;  
            break;  
        case KeyEvent.VK_RIGHT:  
            dx = +5;  
            break;  
    }  
}
```

The values of `dx` and `dy` determine how much the sprite moves each time `step` is invoked. While the user holds down an arrow key, the sprite will move at a constant speed.

Here's an implementation of `keyReleased` that runs when the user releases the key:

```
public void keyReleased(KeyEvent e) {  
    switch (e.getKeyCode()) {  
        case KeyEvent.VK_UP:  
        case KeyEvent.VK_DOWN:  
            dy = 0;  
            break;  
        case KeyEvent.VK_LEFT:  
        case KeyEvent.VK_RIGHT:  
            dx = 0;  
            break;  
    }  
}
```

When the user releases the key, `keyReleased` sets `dx` or `dy` to 0, so the sprite stops moving in that direction.

We don't need the `keyTyped` method for this example, but it's required by the interface; if we don't provide one, the compiler will complain. So we provide an implementation that does nothing:

```
public void keyTyped(KeyEvent e) {  
    // do nothing  
}
```

Now, here's the code we need to create a `Sprite`, add it to a `Drawing`, and configure it as a `KeyListener`:

```
Sprite sprite = new Sprite("face-smile.png", 25, 150);  
drawing.add(sprite);  
drawing.addKeyListener(sprite);  
drawing.setFocusable(true);
```

Recall that the `add` method is one that we wrote in Section ???. It adds an `Actor` to the list of objects to be drawn.

The `addKeyListener` method is inherited from `Canvas`. It adds a `KeyListener` to the list of objects that will receive key events.

In graphical applications, key events are sent to components only when they have the keyboard focus. The `setFocusable` method ensures that `drawing` will have the focus initially, without the user having to click it first.

17.9 Timers

Now that you know about interfaces and events, we can show you a better way to create animations. Previously, we implemented the animation loop by using `while (true)` and `Thread.sleep`. Java provides a `Timer` class (in `javax.swing`) that encapsulates this behavior.

A `Timer` is useful for executing code at regular intervals. The constructor for `Timer` takes two parameters:

- `int` delay // milliseconds between events
- `ActionListener` listener // for handling timer events

The `ActionListener` interface requires only one method, `actionPerformed`. This is the method the `Timer` invokes after the given delay.

Using a `Timer`, we can reorganize the code in `main` by defining a class that implements `ActionListener`:

```
public class VideoGame implements ActionListener {
    private Drawing drawing;

    public VideoGame() {
        Sprite sprite = new Sprite("face-smile.png", 50, 50);
        drawing = new Drawing(800, 600);
        drawing.add(sprite);
        drawing.addKeyListener(sprite);
        drawing.setFocusable(true);

        JFrame frame = new JFrame("Video Game");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.add(drawing);
        frame.pack();
        frame.setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        drawing.step();
    }

    public static void main(String[] args) {
        VideoGame game = new VideoGame();
        Timer timer = new Timer(33, game);
        timer.start();
    }
}
```

The `main` method constructs a `VideoGame` object, which creates a `Sprite`, a `Drawing`, and a `JFrame`. Then it constructs a `Timer` object and starts the timer. Every 33 milliseconds, the `Timer` invokes `actionPerformed`, which invokes `step` on the `Drawing`.

`Drawing.step` invokes `step` on all of its `Actor` objects, which causes them

to update their position, color, or other aspects of their appearance. The `Drawing.step` then repaints the `Canvas`, and the time step is done.

At this point, you have all of the elements you need to write your own video games. In the exercises at the end of this chapter, we have some suggestions for getting started.

We hope this final chapter has been a helpful summary of topics presented throughout the book, including input and output, decisions and loops, classes and methods, arrays and objects, inheritance, and graphics. Congratulations on making it to the end!

17.10 Vocabulary

generalization: The process of extracting common code from two or more classes and moving it into a superclass.

specialization: Extending a class to add new attributes or methods, or to modify existing behavior.

polymorphism: A language feature that allows objects to be assigned to variables of related types.

sprite: A computer graphic that may be moved or otherwise manipulated on the screen.

17.11 Exercises

The code for this chapter is in the `ch17` directory of *ThinkJavaCode2*. See page x for instructions on how to download the repository. Before you start the exercises, we recommend that you compile and run the examples.

The following exercises give you a chance to practice using the features in this chapter by extending the example code.

Exercise 17.1 The `Polygon` class does not provide a `toString` method; it inherits the default `toString` from `java.lang.Object`, which includes only the class's name and memory location. Write a more useful `toString` method for `DrawablePolygon` that includes its (x, y) points.

Exercise 17.2 Write a class `MovingPolygon` that extends `RegularPolygon` and implements `Actor`. It should have instance variables `posx` and `posy` that specify its position, and `dx` and `dy` that specify its velocity (and direction). During each time step, it should update its position. If it gets to the edge of the `Drawing`, it should reverse direction by changing the sign of `dx` or `dy`.

Exercise 17.3 Modify the `VideoGame` class so it displays a `Sprite` and a `MovingPolygon` (from the previous exercise). Add code that detects collisions between `Actor` objects in the same `Drawing`, and invoke a method on both objects when they collide.

Hint: You might want to add a method to the `Actor` interface, guaranteeing that all `Actor` objects know how to handle collisions.

Exercise 17.4 Java provides other event listeners that you can implement to make your programs interactive. For example, the interfaces `MouseListener`, `MouseMotionListener`, and `MouseWheelListener` allow you to respond to mouse input. Use the `MouseListener` interface to implement an `Actor` that can respond to mouse clicks.

Appendix A

Tools

The steps for compiling, running, and debugging Java code depend on your development environment and operating system. We avoided putting these details in the main text, because they can be distracting.

Instead, we provide this appendix with a brief introduction to DrJava—an **integrated development environment** (IDE) that is helpful for beginners—and other development tools, including Checkstyle for code quality and JUnit for testing.

A.1 Installing DrJava

The easiest way to start programming in Java is to use a website that compiles and runs Java code in the browser. Examples include <https://repl.it/>, <https://trinket.io/>, <https://jdoodle.com/>, and others.

If you are unable to install software on your computer (which is often the case in public schools and Internet cafés), you can use these online development environments for almost everything in this book.

But if you want to compile and run Java programs on your own computer, you will need the following:

- The **Java Development Kit** (JDK), which includes the compiler, the **Java Virtual Machine** (JVM) that interprets the compiled byte code, and other tools such as Javadoc.
- A **text editor** such as Atom, Notepad++, or Sublime Text, and/or an IDE such as DrJava, Eclipse, jGrasp, or NetBeans.

The JDK we recommend is OpenJDK, an open source implementation of Java SE (Standard Edition). The IDE we recommend is DrJava, which is an open source development environment written in Java (see Figure ??).

To install OpenJDK, visit <https://adoptopenjdk.net>. Download and run the installer for your operating system.

To install DrJava, visit <http://drjava.org/> and download the **JAR** file. We recommend that you save it to your Desktop folder or another convenient location. Simply double-click the JAR file to run DrJava. Refer to the DrJava documentation (<http://drjava.org/docs/quickstart/>) for more details.

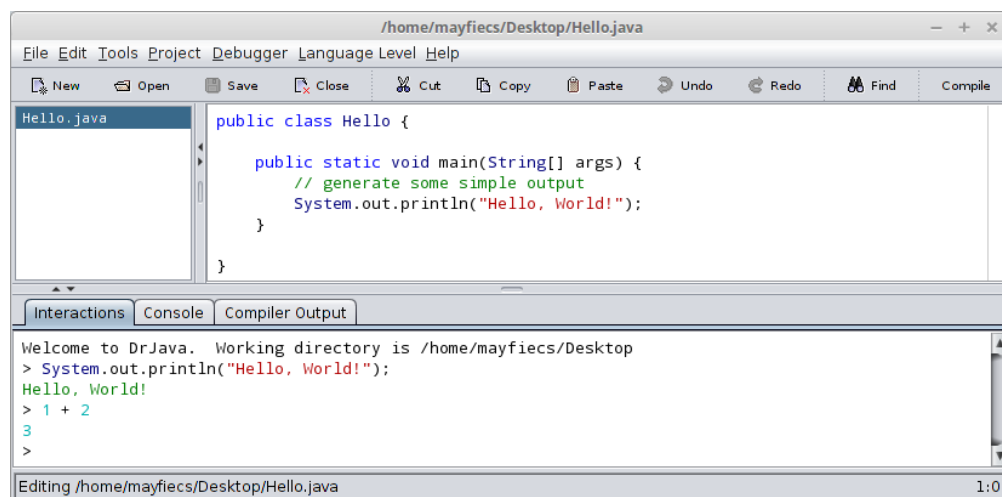


Figure A.1: DrJava editing the Hello World program.

When running DrJava for the first time, we recommend you change three settings from the **Edit > Preferences** menu under **Miscellaneous**: set the **Indent Level** to 4, check the **Automatically Close Block Comments** box, and uncheck the **Keep Emacs-style Backup Files** box.

A.2 DrJava Interactions

One of the most useful features of DrJava is the “Interactions” pane at the bottom of the window. It provides the ability to try out code quickly, without having to write a class definition and save/compile/run the program. Figure ?? shows an example.

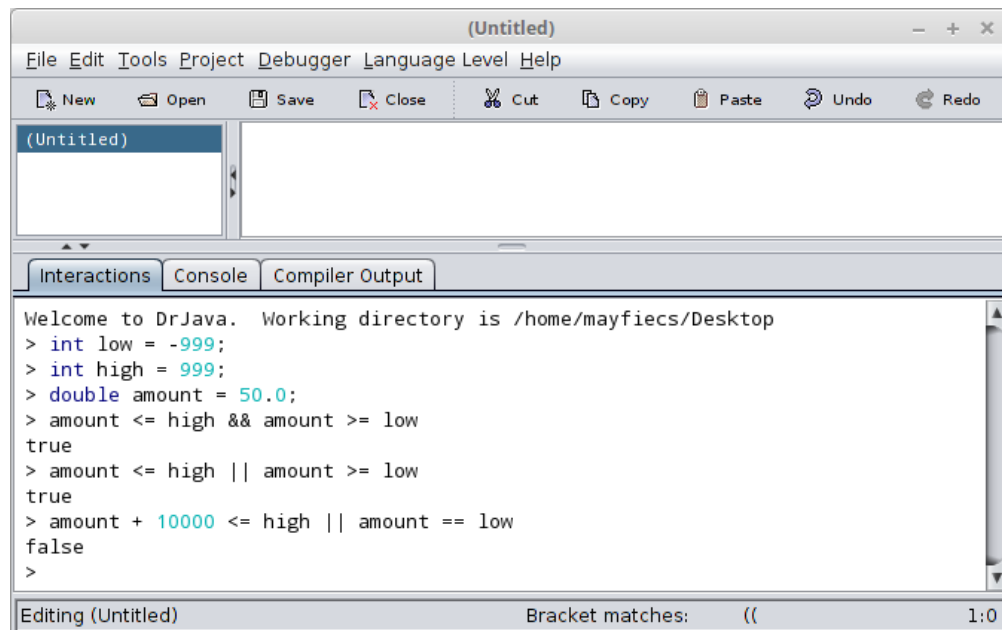


Figure A.2: The Interactions pane in DrJava.

There is one subtle detail to note when using the Interactions pane. If you don’t end an expression (or statement) with a semicolon, DrJava automatically displays its value. Notice in Figure ?? that the variable declarations end with semicolons, but the logic expressions in the following lines do not. This feature saves you from having to type `System.out.println` every time.

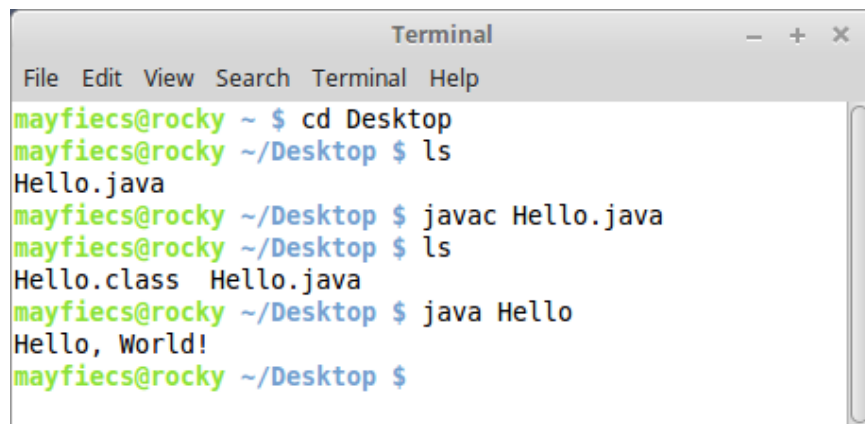
What’s nice about this feature is that you don’t have to create a new class, declare a `main` method, write arbitrary expressions inside `System.out.println` statements, save the source file, and get all of your code to compile in advance. Also, you can press the up/down arrows on the keyboard to repeat previous commands and experiment with incremental differences.

A.3 Command-Line Interface

One of the most powerful and useful skills you can learn is how to use the **command-line interface**, also called the “terminal”. The command line is a direct interface to the operating system. It allows you to run programs, manage files and directories, and monitor system resources. Many advanced tools, both for software development and general-purpose computing, are available only at the command line.

Many good tutorials are available online for learning the command line for your operating system; just search the web for “command line tutorial”. On Unix systems like Linux and macOS, you can get started with just four commands: change the working directory (`cd`), list directory contents (`ls`), compile Java programs (`javac`), and run Java programs (`java`).

Figure ?? shows an example in which the *Hello.java* source file is stored in the *Desktop* directory. After changing to that location and listing the files, we use the `javac` command to compile *Hello.java*. Running `ls` again, we see that the compiler generated a new file, *Hello.class*, which contains the byte code. We run the program by using the `java` command, which displays the output on the following line.

A screenshot of a macOS Terminal window titled "Terminal". The window has a menu bar with "File", "Edit", "View", "Search", "Terminal", and "Help". The terminal shows a series of commands and their outputs. The user is at the prompt "mayfiecs@rocky ~". They enter "cd Desktop", then "ls", which shows "Hello.java". They then enter "javac Hello.java", followed by another "ls" command, which shows "Hello.class" and "Hello.java". Finally, they enter "java Hello", and the output "Hello, World!" is displayed. The prompt is now "mayfiecs@rocky ~/Desktop \$".

```
Terminal
File Edit View Search Terminal Help
mayfiecs@rocky ~ $ cd Desktop
mayfiecs@rocky ~/Desktop $ ls
Hello.java
mayfiecs@rocky ~/Desktop $ javac Hello.java
mayfiecs@rocky ~/Desktop $ ls
Hello.class  Hello.java
mayfiecs@rocky ~/Desktop $ java Hello
Hello, World!
mayfiecs@rocky ~/Desktop $
```

Figure A.3: Compiling and running *Hello.java* from the command line.

Note that the `javac` command requires a *filename* (or multiple source files separated by spaces), whereas the `java` command requires a single *class name*. If you use DrJava, it runs these commands for you behind the scenes and displays the output in the Interactions pane.

Taking time to learn this efficient and elegant way of interacting with the operating system will make you more productive. People who don't use the command line don't know what they're missing.

A.4 Command-Line Testing

As described in Section 1.9, it's more effective to program and debug your code little by little than to attempt writing everything all at once. And after you've completed programming an algorithm, it's important to test that it works correctly on a variety of inputs.

Throughout the book, we illustrate techniques for testing your programs. Most, if not all, testing is based on a simple idea: does the program do what we expect it to do? For simple programs, it's not difficult to run them several times and see what happens. But at some point, you will get tired of typing the same test cases over and over.

We can automate the process of entering input and comparing *expected output* with *actual output* using the command line. The basic idea is to store the test cases in plain text files and trick Java into thinking they are coming from the keyboard. Here are step-by-step instructions:

1. Make sure you can compile and run the *Convert.java* example in the *ch03* directory of *ThinkJavaCode2*. (See page x for instructions on how to download the repository.)
2. In the same directory as *Convert.java*, create a plain text file named *test.in* (“in” is for “input”). Enter the following line and save the file:

```
193.04
```

3. Create a second plain text file named *test.exp* (“exp” is for “expected”). Enter the following line and save the file:

```
193.04 cm = 6 ft, 4 in
```

4. Open a terminal, and change to the directory with these files. Run the following command to test the program:

```
java Convert < test.in > test.out
```

On the command line, `<` and `>` are **redirection operators**. The first one redirects the contents of *test.in* to `System.in`, as if it were entered from the keyboard. The second one redirects the contents of `System.out` to a new file *test.out*, much like a screen capture. In other words, the *test.out* file contains the output of your program.

By the way, it's perfectly okay to compile your programs in DrJava (or another environment) and run them from the command line. Knowing both techniques allows you to use the right tool for the job.

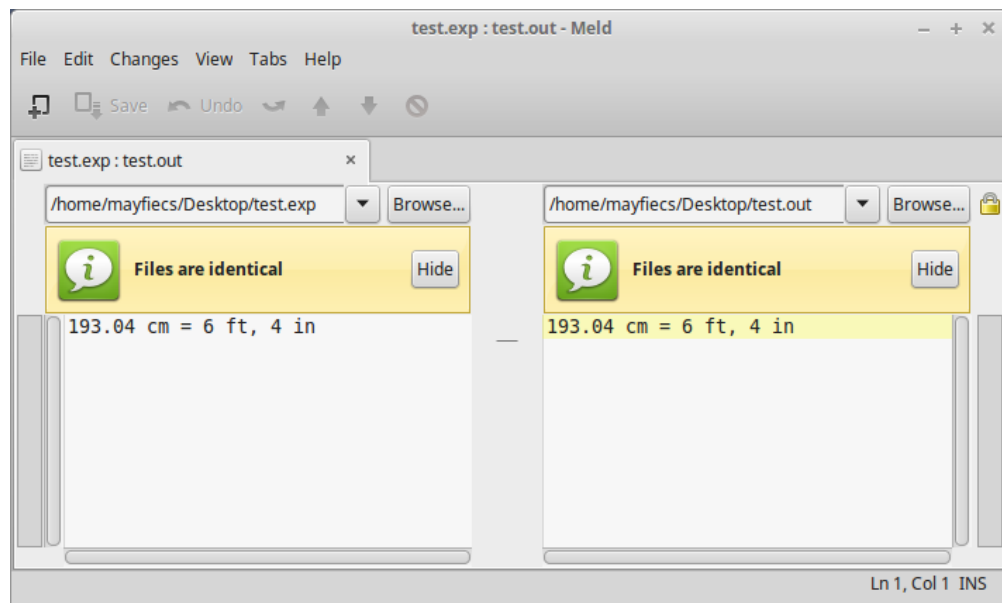


Figure A.4: Using `meld` to compare expected output with the actual output.

At this point, we just need to compare the contents `test.out` with `test.exp`. If the files are the same, then the program outputted what we expected it to output. If not, then we found a bug, and we can use the output to begin debugging our program. Fortunately, there's a simple way to compare files on the command line:

```
diff test.exp test.out
```

The `diff` utility summarizes the differences between two files. If there are no differences, it displays nothing, which in our case is what we want. If the expected output differs from the actual output, we need to continue debugging. Usually, the program is at fault, and `diff` provides some insight about what

is broken. But there's also a chance that we have a correct program and the expected output is wrong.

Interpreting the results from `diff` can be confusing, but fortunately many graphical tools can show the differences between two files. For example, on Windows you can install `WinMerge`, on macOS you can use `opendiff` (which comes with Xcode), and on Linux there's `meld`, shown in Figure ??.

Regardless of what tool you use, the goal is the same. Debug your program until the actual output is *identical* to the expected output.

A.5 Running Checkstyle

Checkstyle is a command-line tool that can be used to determine if your source code follows a set of style rules. It also checks for common programming mistakes, such as class and method design problems.

You can download the latest version as a JAR file from <https://checkstyle.sourceforge.io/>. To run Checkstyle, move (or copy) the JAR file to the same directory as your program. Open a terminal in that location, and run the following command:

```
java -jar checkstyle-*-all.jar -c /google_checks.xml *.java
```

The `*` characters are **wildcards** that match whatever version of Checkstyle you have and whatever Java source files are present. The output indicates the file and line number of each problem. This example refers to a method beginning on line 93, column 5 of *Hello.java*:

```
Hello.java:93:5: Missing a Javadoc comment
```

The file `/google_checks.xml` is inside the JAR file and represents most of Google's style rules. You can alternatively use `/sun_checks.xml` or provide your own configuration file. See Checkstyle's website for more information.

If you apply Checkstyle to your source code often, you will likely internalize good style habits over time. But there are limits to what automatic style checkers can do. In particular, they can't evaluate the *quality* of your comments, the *meaning* of your variable names, or the *structure* of your algorithms.

Good comments make it easier for experienced developers to identify errors in your code. Good variable names communicate the intent of your program and how the data is organized. And good programs are designed to be efficient and demonstrably correct.

A.6 Tracing with a Debugger

A great way to visualize the flow of execution, including how parameters and arguments work, is to use a **debugger**. Most debuggers make it possible to do the following:

- Set a **breakpoint**, a line where you want the program to pause.
- Step through the code one line at a time and watch what it does.
- Check the values of variables and see when and how they change.

For example, open any program in DrJava and move the cursor to the first line of `main`. Press `Ctrl+B` to toggle a breakpoint on the current line; it should now be highlighted in red. Press `Ctrl+Shift+D` to turn on “Debug Mode”; a new pane should appear at the bottom of the window. These commands are also available from the **Debugger** menu, in case you forget the shortcut keys.

When you run the program, execution pauses at the first breakpoint. The debugging pane displays the **call stack**, with the current method on top of the stack, as shown in Figure ???. You might be surprised to see how many methods were called before the `main` method!

To the right are several buttons that allow you to step through the code at your own pace. You can also click **Automatic Trace** to watch DrJava run your code one line at a time.

Using a debugger is like having the computer proofread your code out loud. When the program is paused, you can examine (or even change) the value of any variable by using the Interactions pane.

Tracing allows you to follow the flow of execution and see how data passes from one method to another. You might expect the code do one thing, but

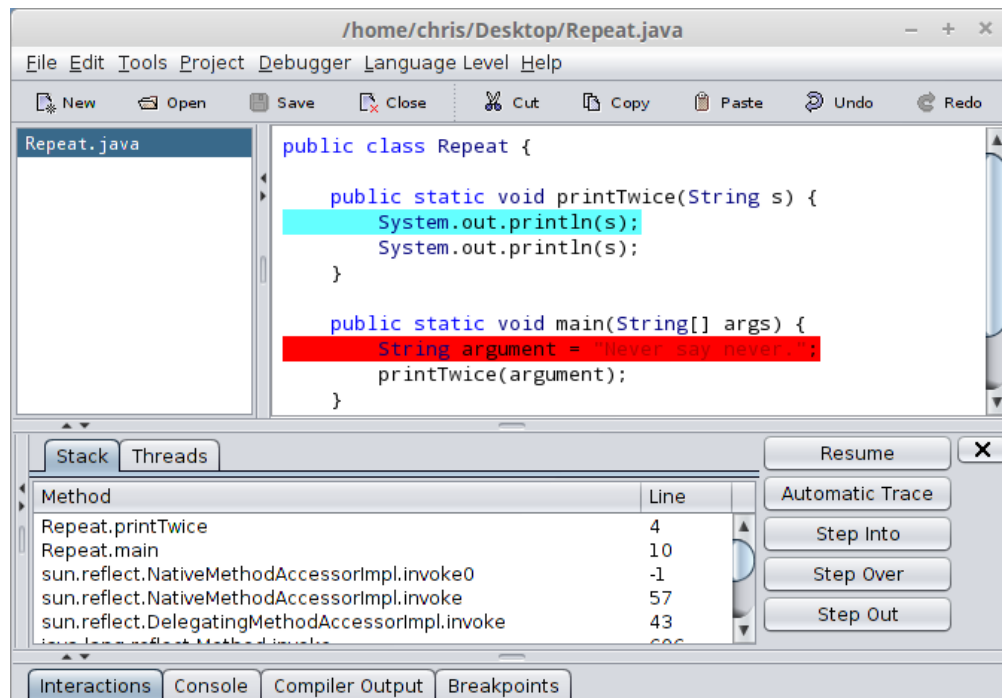


Figure A.5: The DrJava debugger. Execution is currently paused on the first line of `printTwice`. There is a breakpoint on the first line of `main`.

then the debugger shows it doing something else. At that moment, you gain insight about what may be wrong with the code.

You can edit your code while debugging it, but we don't recommend it. If you add or delete multiple lines of code while the program is paused, the results can be confusing.

See <http://drjava.org/docs/user/ch09.html> for more information about using the debugger feature of DrJava.

A.7 Testing with JUnit

When beginners start writing methods, they usually test them by invoking them from `main` and checking the results by hand. For example, to test `fibonacci` from Section ??, we could write this:

```
public static void main(String[] args) {
    if (fibonacci(1) != 1) {
        System.err.println("fibonacci(1) is incorrect");
    }
    if (fibonacci(2) != 1) {
        System.err.println("fibonacci(2) is incorrect");
    }
    if (fibonacci(3) != 2) {
        System.err.println("fibonacci(3) is incorrect");
    }
}
```

This test code is self-explanatory, but it's longer than it needs to be, and it doesn't scale very well. In addition, the error messages provide limited information. For cases where we know the right answer, we can do better by writing **unit tests**.

JUnit (<https://junit.org/>) is a common testing tool for Java programs. To use it, you have to create a test class that contains test methods.

For example, suppose that the `fibonacci` method belongs to a class named `Series`. Here is a corresponding JUnit¹ test class and test method:

```
import junit.framework.TestCase;

public class SeriesTest extends TestCase {

    public void testFibonacci() {
        assertEquals(1, Series.fibonacci(1));
        assertEquals(1, Series.fibonacci(2));
        assertEquals(2, Series.fibonacci(3));
    }
}
```

This example uses the keyword `extends`, which indicates that the new class, `SeriesTest`, is based on an existing class, `TestCase`. The `TestCase` class is imported from the package `junit.framework`.

¹This code is based on JUnit 3, which has been superseded but is the default version supported by DrJava.

The names in this example follow convention: if the name of your class is `Something`, the name of the test class should be `SomethingTest`. And if there is a method in `Something` named `someMethod`, there should be a method in `SomethingTest` named `testSomeMethod`.

Many development environments can generate test classes and test methods automatically. In DrJava, you can select **New JUnit Test Case** from the File menu to generate an empty test class.

`assertEquals` is provided by the `TestCase` class. It takes two arguments and checks whether they are equal. If so, it does nothing; otherwise, it displays a detailed error message. The first argument is the *expected value*, which we consider correct, and the second argument is the *actual value* we want to check. If they are not equal, the test fails.

Using `assertEquals` is more concise than writing your own `if` statements and `System.err` messages. JUnit provides additional assert methods, such as `assertNull`, `assertSame`, and `assertTrue`, which can be used to design a variety of tests.

To run JUnit directly from DrJava, click the **Test** button on the toolbar. If all your test methods pass, you will see a green bar in the lower-right corner. Otherwise, DrJava will take you directly to the first assertion that failed.

A.8 Vocabulary

IDE: An “integrated development environment” that includes tools for editing, compiling, and debugging programs.

JDK: The “Java Development Kit”, which contains the compiler, Javadoc, and other tools.

JVM: The “Java Virtual Machine”, which interprets the compiled byte code.

text editor: A program that edits plain text files, the format used by most programming languages.

JAR: A “Java Archive”, which is essentially a ZIP file containing classes and other resources.

command-line interface: A means of interacting with the computer by issuing commands in the form of successive lines of text.

redirection operator: A command-line feature that substitutes `System.in` and/or `System.out` with a plain text file.

wildcard: A command-line feature that allows you to specify a pattern of filenames by using the `*` character.

debugger: A tool that allows you to run one statement at a time and see the contents of variables.

breakpoint: A line of code at which the debugger will pause a running program.

call stack: The history of method calls and where to resume execution after each method returns.

unit test: Code that exercises a single method of a program, testing for correctness and/or efficiency.

Appendix B

Javadoc

Java programs have three types of comments:

End-of-line comments: These start with `//` and generally contain short phrases that explain specific lines of code.

Multiline comments: These start with `/*` and end with `*/`, and are typically used for copyright statements.

Documentation comments: These start with `/**` and end with `*/`, and describe what each class and method does.

End-of-line and multiline comments are written primarily for yourself. They help you remember specific details about your source code. Documentation comments, on the other hand, are written for others. They explain how to use your classes and methods in other programs.

A nice feature of the Java language is the ability to embed documentation in the source code itself. That way, you can write it as you go, and as things change, it is easier to keep the documentation consistent with the code.

You can extract documentation from your source code, and generate well-formatted HTML pages, using a tool called **Javadoc**. This tool is included with the JDK, and it is widely used. In fact, the official documentation for the Java library (<https://thinkjava.org/apidoc>) is generated by Javadoc.

B.1 Reading Documentation

As an example, let's look at the documentation for `Scanner`, a class we first used in Section 3.2. You can find the documentation quickly by doing a web search for “Java Scanner”. Figure ?? shows a screenshot of the page.

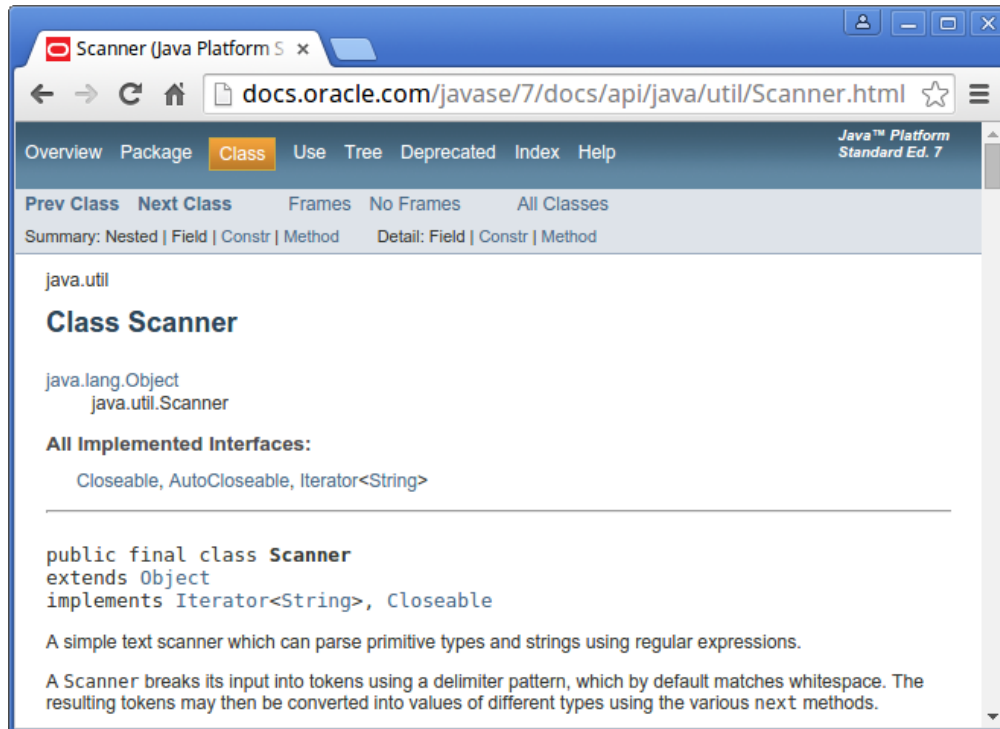


Figure B.1: The documentation for `Scanner`.

Documentation for other classes uses a similar format. The first line is the package that contains the class, such as `java.util`. The second line is the name of the class. The “All Implemented Interfaces” section lists some of the functionality a `Scanner` has.

The next section of the documentation is a narrative that explains the purpose of the class and includes examples of how to use it. This text can be difficult to read, because it may use terms you have not yet learned. But the examples are often very useful. A good way to get started with a new class is to paste the examples into a test file and see if you can compile and run them.

One of the examples shows how you can use a `Scanner` to read input from a `String` instead of `System.in`:

```
String input = "1 fish 2 fish red fish blue fish";  
Scanner s = new Scanner(input);
```

After the narrative, code examples, and other details, you will find the following tables:

Constructor Summary: Ways of creating, or constructing, a `Scanner`.

Method Summary: The list of methods that the `Scanner` class provides.

Constructor Detail: More information about how to create a `Scanner`.

Method Detail: More information about each method.

For example, here is the summary information for `nextInt`:

```
public int nextInt()  
Scans the next token of the input as an int.
```

The first line is the method's **signature**, which specifies the name of the method and its parameters (none). The type it returns (`int`) is not part of the signature. The next line is a short description of what it does.

The “Method Detail” explains more:

```
public int nextInt()  
Scans the next token of the input as an int.
```

An invocation of this method of the form `nextInt()` behaves in exactly the same way as the invocation `nextInt(radix)`, where `radix` is the default radix of this scanner.

Returns:
the int scanned from the input

Throws:
`InputMismatchException` - if the next token does not match
the Integer regular expression, or is out of range
`NoSuchElementException` - if input is exhausted
`IllegalStateException` - if this scanner is closed

The “Returns” section describes the result when the method succeeds. In contrast, the “Throws” section describes possible errors and exceptions that may occur. Exceptions are said to be thrown, like a referee throwing a flag, or like a toddler throwing a fit.

It might take you some time to get comfortable reading documentation and learning which parts to ignore. But it’s worth the effort. Knowing what’s available in the library helps you avoid reinventing the wheel. And a little bit of documentation can save you a lot of debugging.

B.2 Writing Documentation

As you benefit from reading good documentation, you should “pay it forward” by writing good documentation.

Javadoc scans your source files looking for documentation comments, also known as “Javadoc comments”. They begin with `/**` (two stars) and end with `*/` (one star). Anything in between is considered part of the documentation.

Here’s a class definition with two Javadoc comments, one for the `Goodbye` class and one for the `main` method:

```
/**
 * Example program that demonstrates print vs println.
 */
public class Goodbye {

    /**
     * Prints a greeting.
     */
    public static void main(String[] args) {
        System.out.print("Goodbye, "); // note the space
        System.out.println("cruel world");
    }
}
```

The class comment explains the purpose of the class. The method comment explains what the method does.

Notice that this example also has an end-of-line comment (`/**`). In general, these comments are short phrases that help explain complex parts of a program. They are intended for other programmers reading and maintaining the source code.

In contrast, Javadoc comments are longer, usually complete sentences. They explain what each method does, but they omit details about how the method works. And they are intended for people who will use the methods without looking at the source code.

Appropriate comments and documentation are essential for making source code readable. And remember that the person most likely to read your code in the future, and appreciate good documentation, is you.

B.3 Javadoc Tags

It's generally a good idea to document each class and method, so that other programmers can understand what they do without having to read the code.

To organize the documentation into sections, Javadoc supports optional **tags** that begin with the at sign (`@`). For example, we can use `@author` and `@version` to provide information about the class:

```
/**
 * Utility class for extracting digits from integers.
 *
 * @author Chris Mayfield
 * @version 1.0
 */
public class DigitUtil {
```

Documentation comments should begin with a **description** of the class or method, followed by the tags. These two sections are separated by a blank line (not counting the `*`).

For methods, we can use `@param` and `@return` to provide information about parameters and return values:

```
/**
 * Tests whether x is a single digit integer.
 *
 * @param x the integer to test
 * @return true if x has one digit, false otherwise
 */
public static boolean isSingleDigit(int x) {
```

Figure ?? shows part of the resulting HTML page generated by Javadoc. Notice the relationship between the Javadoc comment (in the source code) and the resulting documentation (in the HTML page).

isSingleDigit
<pre>public static boolean isSingleDigit(int x)</pre>
Tests whether x is a single digit integer.
Parameters:
x - the integer to test
Returns:
true if x has one digit, false otherwise

Figure B.2: HTML documentation for `isSingleDigit`.

When writing parameter comments, do not include a hyphen (-) after the `@param` tag. Otherwise, you will have two hyphens in the resulting HTML documentation.

Notice also that the `@return` tag should not specify the type of the method. Comments like `@return boolean` are not useful, because you already know the return type from the method's signature.

Methods with multiple parameters should have separate `@param` tags that describe each one. Void methods should have no `@return` tag, since they do not return a value. Each tag should be on its own line in the source code.

B.4 Example Source File

Now let's take a look at a more complete example. The code for this section is in the *appb* directory of *ThinkJavaCode2*. See page x for instructions on how to download the repository.

Professional-grade source files often begin with a copyright statement. This text spans multiple lines, but it is not part of the documentation. So we use a multiline comment (*/**) rather than a documentation comment (*/***). Our example source file, *Convert.java*, includes the MIT License (<https://opensource.org/licenses/MIT>):

```
/*
 * Copyright (c) 2019 Allen Downey and Chris Mayfield
 *
 * Permission is hereby granted, free of charge, to any person obtaining a copy
 * of this software and associated documentation files (the "Software"), to deal
 * in the Software without restriction, including without limitation the rights
 * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
 * copies of the Software, and to permit persons to whom the Software is
 * furnished to do so, subject to the following conditions:
 *
 * The above copyright notice and this permission notice shall be included in
 * all copies or substantial portions of the Software.
 *
 * THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
 * IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
 * FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
 * AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
 * LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
 * OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
 * SOFTWARE.
 */
```

Import statements generally follow the copyright text. After that, we can define the class itself and begin writing the documentation (*/***):

```
import java.util.Scanner;

/**
 * Methods for converting to/from the metric system.
 *
 * @author Allen Downey
 * @author Chris Mayfield
 * @version 6.1.5
 */
public class Convert {
```

A common mistake that beginners make is to put `import` statements between the documentation and the `public class` line. Doing so separates the documentation from the class itself. To avoid this issue, always make the end of the comment (the `*/`) “touch” the word `public`.

This class has two constants and three methods. The constants are self-explanatory, so there is no need to write documentation for them:

```
public static final double CM_PER_INCH = 2.54;

public static final int IN_PER_FOOT = 12;
```

The methods, on the other hand, could use some explanation. Each documentation comment includes a description, followed by a blank line, followed by a `@param` tag for each parameter, followed by a `@return` tag:

```
/**
 * Converts a measurement in centimeters to inches.
 *
 * @param cm length in centimeters
 * @return length in inches
 */
public static double toImperial(double cm) {
    return cm / CM_PER_INCH;
}

/**
 * Converts a length in feet and inches to centimeters.
 *
 * @param feet how many feet
 * @param inches how many inches
 * @return length in centimeters
 */
public static double toMetric(int feet, int inches) {
    int total = feet * IN_PER_FOOT + inches;
    return total * CM_PER_INCH;
}
```

The main method has a similar documentation comment, except there is no `@return` tag since the method is `void`:

```
/**
 * Tests the conversion methods.
 *
 * @param args command-line arguments
 */
public static void main(String[] args) {
    double cm, result;
    int feet, inches;
    Scanner in = new Scanner(System.in);

    // test the Imperial conversion
    System.out.print("Exactly how many cm? ");
    cm = in.nextDouble();
    result = toImperial(cm);
    System.out.printf("That's %.2f inches\n", result);
    System.out.println();

    // test the Metric conversion
    System.out.print("Now how many feet? ");
    feet = in.nextInt();
    System.out.print("And how many inches? ");
    inches = in.nextInt();
    result = toMetric(feet, inches);
    System.out.printf("That's %.2f cm\n", result);
}
```

Here are two ways you can run the Javadoc tool on this example program:

- From the command line, go to the location for *Convert.java*. The `-d` option of `javadoc` indicates where to generate the HTML files:

```
javadoc -d doc Convert.java
```

- From DrJava, click the **Javadoc** button on the toolbar. The IDE will then prompt you for a location to generate the HTML files.

For more examples of what you can do with Javadoc comments, see the source code of any Java library class (e.g., *Scanner.java*). Section ?? explains how to find the source files for the Java library on your computer.

B.5 Vocabulary

documentation: Comments that describe the technical operation of a class or method.

Javadoc: A tool that reads Java source code and generates documentation in HTML format.

signature: The first line of a method that defines its name and parameters.

tag: A label that begins with an at sign (@) and is used by Javadoc to organize documentation into sections.

description: The first line of a documentation comment that explains what the class/method does.

Appendix C

Graphics

The Java library includes the package `java.awt` for drawing 2D graphics. **AWT** stands for “Abstract Window Toolkit”. We are only going to scratch the surface of graphics programming. You can read more about it in the Java tutorials (see <https://thinkjava.org/java2d>).

C.1 Creating Graphics

There are several ways to create graphics in Java; the simplest way is to use `java.awt.Canvas` and `java.awt.Graphics`. A `Canvas` is a blank rectangular area of the screen onto which the application can draw. The `Graphics` class provides basic drawing methods such as `drawLine`, `drawRect`, and `drawString`.

Here is an example program that draws a circle by using the `fillOval` method:

```
import java.awt.Canvas;
import java.awt.Graphics;
import javax.swing.JFrame;

public class Drawing extends Canvas {
```

```
public static void main(String[] args) {
    JFrame frame = new JFrame("My Drawing");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    Drawing drawing = new Drawing();
    drawing.setSize(400, 400);
    frame.add(drawing);
    frame.pack();
    frame.setVisible(true);
}

public void paint(Graphics g) {
    g.fillOval(100, 100, 200, 200);
}
}
```

The `Drawing` class extends `Canvas`, so it has all the methods provided by `Canvas`, including `setSize`. You can read about the other methods in the documentation, which you can find by doing a web search for “Java Canvas”.

In the `main` method, we do the following:

1. Create a `JFrame` object, which is the window that will contain the canvas.
2. Create a `Drawing` object (which is the canvas), set its width and height, and add it to the frame.
3. Pack the frame (resize it) to fit the canvas, and display it on the screen.

Once the frame is visible, the `paint` method is called whenever the canvas needs to be drawn; for example, when the window is moved or resized. If you run this code, you should see a black circle on a gray background.

The application doesn’t end after the `main` method returns; instead, it waits for the `JFrame` to close. When the `JFrame` closes, it calls `System.exit`, which ends the program.

C.2 Graphics Methods

You are probably used to Cartesian **coordinates**, where x and y values can be positive or negative. In contrast, Java uses a coordinate system where the

origin is in the upper-left corner. That way, x and y can always be positive integers. Figure ?? shows these coordinate systems side by side.

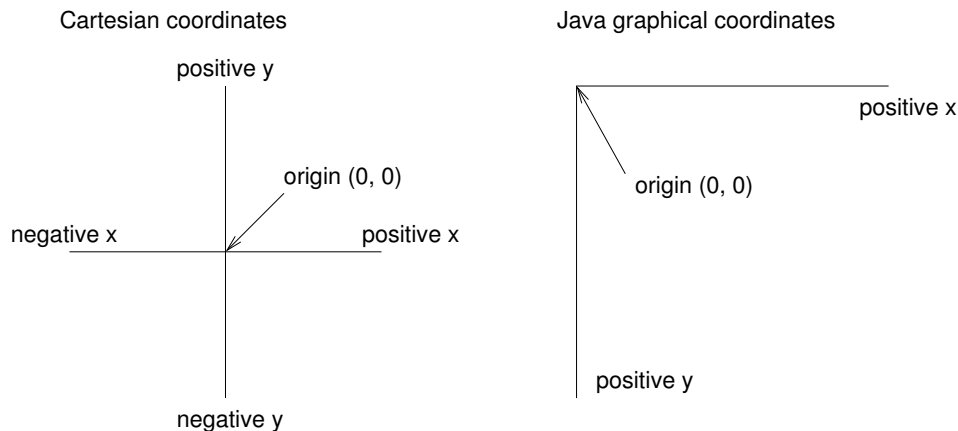


Figure C.1: The difference between Cartesian coordinates and Java graphical coordinates.

Graphical coordinates are measured in **pixels**; each pixel corresponds to a dot on the screen.

To draw on the canvas, you invoke methods on a **Graphics** object. You don't have to create the **Graphics** object; it gets created when you create the **Canvas**, and it gets passed as an argument to **paint**.

The previous example used **fillOval**, which has the following signature:

```
/**
 * Fills an oval bounded by the specified rectangle with
 * the current color.
 */
public void fillOval(int x, int y, int width, int height)
```

The four parameters specify a **bounding box**, which is the rectangle in which the oval is drawn. **x** and **y** specify the location of the upper-left corner of the bounding box. The bounding box itself is not drawn (see Figure ??).

To choose the color of a shape, invoke **setColor** on the **Graphics** object:

```
g.setColor(Color.RED);
```

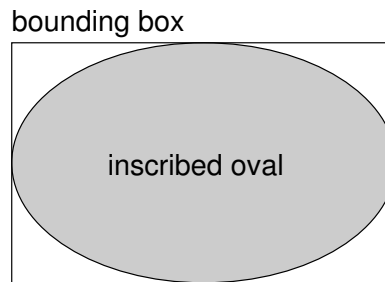


Figure C.2: An oval inside its bounding box.

The `setColor` method determines the color of everything that gets drawn afterward. `Color.red` is a constant provided by the `Color` class; to use it, you have to `import java.awt.Color`. Other colors include the following:

BLACK	BLUE	CYAN	DARKGRAY	GRAY	LIGHTGRAY
GREEN	MAGENTA	ORANGE	PINK	WHITE	YELLOW

You can create your own colors by specifying the red, green, and blue (**RGB**) components. For example:

```
Color purple = new Color(128, 0, 128);
```

Each value is an integer in the range 0 (darkest) to 255 (lightest). The color (0, 0, 0) is black, and (255, 255, 255) is white.

You can set the background color of the `Canvas` by invoking `setBackground`:

```
canvas.setBackground(Color.WHITE);
```

C.3 Example Drawing

Suppose we want to draw a “Hidden Mickey”, which is an icon that represents Mickey Mouse (see https://en.wikipedia.org/wiki/Hidden_Mickey). We can use the oval we just drew as the face, and then add two ears. To make the code more readable, let’s use `Rectangle` objects to represent bounding boxes.

Here’s a method that takes a `Rectangle` and invokes `fillOval`:


```
public void boxOval(Graphics g, Rectangle bb) {  
    g.fillOval(bb.x, bb.y, bb.width, bb.height);  
}
```

And here's a method that draws Mickey Mouse:

```
public void mickey(Graphics g, Rectangle bb) {  
    boxOval(g, bb);  
  
    int hx = bb.width / 2;  
    int hy = bb.height / 2;  
    Rectangle half = new Rectangle(bb.x, bb.y, hx, hy);  
  
    half.translate(-hx / 2, -hy / 2);  
    boxOval(g, half);  
  
    half.translate(hx * 2, 0);  
    boxOval(g, half);  
}
```

The first line draws the face. The next three lines create a smaller rectangle for the ears. We `translate` the rectangle up and left for the first ear, then to the right for the second ear. The result is shown in Figure ??.

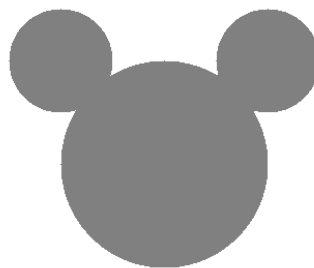


Figure C.3: A Hidden Mickey drawn using Java graphics.

You can read more about `Rectangle` and `translate` in Chapter ?. See the exercises at the end of this appendix for more example drawings.

C.4 Vocabulary

AWT: The “Abstract Window Toolkit”, a Java package for creating graphical user interfaces.

coordinate: A value that specifies a location in a 2D graphical window.

pixel: The unit in which coordinates are measured.

bounding box: A way to specify the coordinates of a rectangular area.

RGB: A color model based on adding red, green, and blue light.

C.5 Exercises

The code for this chapter is in the *appc* directory of *ThinkJavaCode2*. See page x for instructions on how to download the repository. Before you start the exercises, we recommend that you compile and run the examples.

Exercise C.1 Draw the flag of Japan: a red circle on a white background that is wider than it is tall.

Exercise C.2 Modify *Mickey.java* to draw ears on the ears, and ears on those ears, and more ears all the way down until the smallest ears are only 3 pixels wide. The result should look like Figure ?? . *Hint:* You should have to add or modify only a few lines of code.

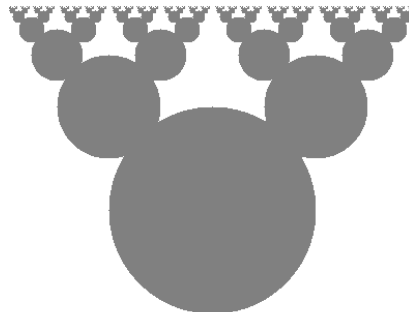
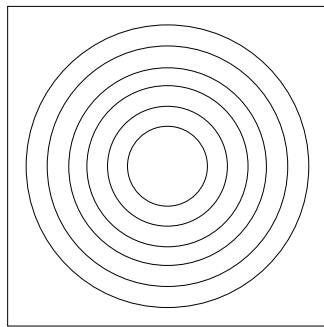


Figure C.4: A recursive shape we call “Mickey Moose”.

Exercise C.3 In this exercise, you will draw “Moiré patterns” that seem to shift around as you move. For an explanation of what is going on, see https://en.wikipedia.org/wiki/Moire_pattern.

1. Open *Moire.java* and read the `paint` method. Draw a sketch of what you expect it to do. Now run it. Did you get what you expected?
2. Modify the program so that the space between the circles is larger or smaller. See what happens to the image.
3. Modify the program so that the circles are drawn in the center of the screen and concentric, as in Figure ?? (left). The distance between the circles should be small enough that the Moiré interference is apparent.

Concentric circles



Radial Moire pattern

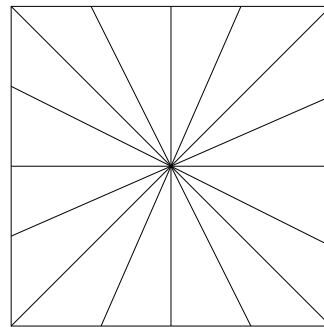


Figure C.5: Graphical patterns that can exhibit Moiré interference.

4. Write a method named `radial` that draws a radial set of line segments as shown in Figure ?? (right), but they should be close enough together to create a Moiré pattern.
5. Just about any kind of graphical pattern can generate Moiré-like interference patterns. Play around and see what you can create.

Appendix D

Debugging

Although there are debugging suggestions throughout the book, we thought it would be useful to say more in an appendix. If you are having a hard time debugging, you might want to review this appendix from time to time.

The best debugging strategy depends on what kind of error you have:

- **Compile-time errors** indicate that there is something wrong with the syntax of the program. Example: omitting the semicolon at the end of a statement.
- **Run-time errors** are produced if something goes wrong while the program is running. Example: an infinite recursion eventually causes a `StackOverflowError`.
- **Logic errors** cause the program to do the wrong thing. Example: an expression may not be evaluated in the order you expect.

The following sections are organized by error type; some techniques are useful for more than one type.

D.1 Compile-Time Errors

The best kind of debugging is the kind you don't have to do because you avoid making errors in the first place. Incremental development, which we presented

in Section 4.9, can help. The key is to start with a working program and add small amounts of code at a time. When there is an error, you will have a pretty good idea of where it is.

Nevertheless, you might find yourself in one of the following situations. For each situation, we have some suggestions about how to proceed.

The compiler is spewing error messages.

If the compiler reports 100 error messages, that doesn't mean there are 100 errors in your program. When the compiler encounters an error, it often gets thrown off track for a while. It tries to recover and pick up again after the first error, but sometimes it reports spurious errors.

Only the first error message is truly reliable. We suggest that you fix only one error at a time and then recompile the program. You may find that one semicolon or brace “fixes” 100 errors.

I'm getting a weird compiler message, and it won't go away.

First of all, read the error message carefully. It may be written in terse jargon, but often there is a carefully hidden kernel of information.

If nothing else, the message will tell you where in the program the problem occurred. Actually, it tells you where the compiler was when it noticed a problem, which is not necessarily where the error is. Use the information the compiler gives you as a guideline, but if you don't see an error where the compiler is pointing, broaden the search.

Generally, the error will be prior to the location of the error message, but in some cases it will be somewhere else entirely. For example, if you get an error message at a method invocation, the actual error may be in the method definition itself.

If you don't find the error quickly, take a breath and look more broadly at the entire program. Make sure the program is indented properly; that makes it easier to spot syntax errors.

Now, start looking for common syntax errors:

1. Check that all parentheses and brackets are balanced and properly nested. All method definitions should be nested within a class definition. All program statements should be within a method definition.
2. Remember that uppercase letters are not the same as lowercase letters.
3. Check for semicolons at the end of statements (and no semicolons after curly braces).
4. Make sure that any strings in the code have matching quotation marks. Make sure that you use double quotes for strings, and single quotes for characters.
5. For each assignment statement, make sure that the type on the left is the same as the type on the right. Make sure that the expression on the left is a variable name or something else that you can assign a value to (like an element of an array).
6. For each method invocation, make sure that the arguments you provide are in the right order and have the right type, and that the object you are invoking the method on is the right type.
7. If you are invoking a value method, make sure you are doing something with the result. If you are invoking a void method, make sure you are *not* trying to do something with the result.
8. If you are invoking an instance method, make sure you are invoking it on an object with the right type. If you are invoking a static method from outside the class where it is defined, make sure you specify the class name (using dot notation).
9. Inside an instance method, you can refer to the instance variables without specifying an object. If you try that in a static method—with or without `this`—you get a message like “non-static variable x cannot be referenced from a static context.”

If nothing works, move on to the next section...

I can't get my program to compile no matter what I do.

If the compiler says there is an error and you don't see it, that might be because you and the compiler are not looking at the same code. Check your development environment to make sure the program you are editing is the program the compiler is compiling.

This situation is often the result of having multiple copies of the same program. You might be editing one version of the file but compiling a different version.

If you are not sure, try putting an obvious and deliberate syntax error right at the beginning of the program. Now compile again. If the compiler doesn't find the new error, there is probably something wrong with the way you set up the development environment.

If you have examined the code thoroughly, and you are sure the compiler is compiling the right source file, it is time for desperate measures—**debugging by bisection**:

- Make a backup of the file you are working on. If you are working on *Bob.java*, make a copy called *Bob.java.old*.
- Delete about half the code from *Bob.java*. Try compiling again.
 - If the program compiles now, you know the error is in the code you deleted. Bring back about half of what you deleted and repeat.
 - If the program still doesn't compile, the error must be in the code that remains. Delete about half of the remaining code and repeat.
- Once you have found and fixed the error, start bringing back the code you deleted, a little bit at a time.

This process is ugly, but it goes faster than you might think and is very reliable. It works for other programming languages too!

I did what the compiler told me to do, but it still doesn't work.

Some error messages come with tidbits of advice, like “class Golfer must be declared abstract. It does not define `compareTo(java.lang.Object)` from

interface java.lang.Comparable.” It sounds like the compiler is telling you to declare `Golfer` as an `abstract` class, and if you are reading this book, you probably don’t know what that is or how to do it.

Fortunately, the compiler is wrong. The solution in this case is to make sure `Golfer` has a method called `compareTo` that takes an `Object` as a parameter.

Don’t let the compiler lead you by the nose. Error messages give you evidence that something is wrong, but the remedies they suggest are unreliable.

D.2 Run-Time Errors

It’s not always clear what causes a run-time error, but you can often figure things out by adding print statements to your program.

My program hangs.

If a program stops and seems to be doing nothing, we say it is “hanging”. Often that means it is caught in an infinite loop or an infinite recursion.

- If you suspect that a particular loop is the problem, add a print statement immediately before the loop that says `"entering the loop"` and another immediately after that says `"exiting the loop"`.

Run the program. If you get the first message and not the second, you know where the program is getting stuck. Go to the section titled “Infinite loop”.

- Most of the time, an infinite recursion will cause the program to run for a while and then produce a `StackOverflowError`. If that happens, go to the section titled “Infinite recursion”.

If you are not getting a `StackOverflowError`, but you suspect there is a problem with a recursive method, you can still use the techniques in the infinite recursion section.

- If neither of the previous suggestions helps, you might not understand the flow of execution in your program. Go to the section titled “Flow of execution”.

Infinite loop

If you think you have an infinite loop and you know which loop it is, add a print statement at the end of the loop that displays the values of the variables in the condition, and the value of the condition.

For example:

```
while (x > 0 && y < 0) {  
    // do something to x  
    // do something to y  
  
    System.out.println("x: " + x);  
    System.out.println("y: " + y);  
    System.out.println("condition: " + (x > 0 && y < 0));  
}
```

Now when you run the program, you see three lines of output for each time through the loop. The last time through the loop, the condition should be **false**. If the loop keeps going, you will see the values of `x` and `y`, and you might figure out why they are not getting updated correctly.

Infinite recursion

Most of the time, an infinite recursion will cause the program to throw a **StackOverflowError**. But if the program is slow, it may take a long time to fill the stack.

If you know which method is causing an infinite recursion, check that there is a base case. There should be a condition that makes the method return without making a recursive invocation. If not, you need to rethink the algorithm and identify a base case.

If there is a base case, but the program doesn't seem to be reaching it, add a print statement at the beginning of the method that displays the parameters.

Now when you run the program, you see a few lines of output every time the method is invoked, and you can see the values of the parameters. If the parameters are not moving toward the base case, you might see why not.

Flow of execution

If you are not sure how the flow of execution is moving through your program, add print statements to the beginning of each method with a message like `"entering method foo"`, where `foo` is the name of the method. Now when you run the program, it displays a trace of each method as it is invoked.

You can also display the arguments each method receives. When you run the program, check whether the values are reasonable, and check for one of the most common errors—providing arguments in the wrong order.

When I run the program, I get an exception.

When an exception occurs, Java displays a message that includes the name of the exception, the line of the program where the exception occurred, and a stack trace. The stack trace includes the method that was running, the method that invoked it, the method that invoked that one, and so on.

The first step is to examine the place in the program where the error occurred and see if you can figure out what happened:

NullPointerException:

You tried to access an instance variable or invoke a method on an object that is currently `null`. You should figure out which variable is `null` and then figure out how it got to be that way.

Remember that when you declare a variable with an array type, its elements are initially `null` until you assign a value to them. For example, this code causes a `NullPointerException`:

```
int[] array = new Point[5];  
System.out.println(array[0].x);
```

ArrayIndexOutOfBoundsException:

The index you are using to access an array is either negative or greater than `array.length - 1`. If you can find the site where the problem is, add a print statement immediately before it to display the value of the index and the length of the array. Is the array the right size? Is the index the right value?

Now work your way backward through the program and see where the array and the index come from. Find the nearest assignment statement and see if it is doing the right thing. If either one is a parameter, go to the place where the method is invoked and see where the values are coming from.

StackOverflowError:

See “Infinite recursion” on page ??.

FileNotFoundException:

This means Java didn’t find the file it was looking for. If you are using a project-based development environment like Eclipse, you might have to import the file into the project. Otherwise, make sure the file exists and that the path is correct. This problem depends on your filesystem, so it can be hard to track down.

ArithmeticException:

Something went wrong during an arithmetic operation; for example, division by zero.

I added so many print statements I get inundated with output.

One of the problems with using print statements for debugging is that you can end up buried in output. There are two ways to proceed: either simplify the output or simplify the program.

To simplify the output, you can remove or comment out print statements that aren’t helping, or combine them, or format the output so it is easier to understand. As you develop a program, you should write code to generate concise, informative traces of what the program is doing.

To simplify the program, scale down the problem the program is working on. For example, if you are sorting an array, sort a *small* array. If the program takes input from the user, give it the simplest input that causes the error.

Also, clean up the code. Remove unnecessary or experimental parts, and reorganize the program to make it easier to read. For example, if you suspect

that the error is in a deeply nested part of the program, rewrite that part with a simpler structure. If you suspect a large method, split it into smaller methods and test them separately.

The process of finding the minimal test case often leads you to the bug. For example, if you find that a program works when the array has an even number of elements, but not when it has an odd number, that gives you a clue about what is going on.

Reorganizing the program can help you find subtle bugs. If you make a change that you think doesn't affect the program, and it does, that can tip you off.

D.3 Logic Errors

My program doesn't work.

Logic errors are hard to find because the compiler and interpreter provide no information about what is wrong. Only you know what the program is supposed to do, and only you know that it isn't doing it.

The first step is to make a connection between the code and the behavior you get. You need a hypothesis about what the program is actually doing. Here are some questions to ask yourself:

- Is there something the program was supposed to do that doesn't seem to be happening? Find the section of the code that performs that function, and make sure it is executing when you think it should. See "Flow of execution" on page ??.
- Is something happening that shouldn't? Find code in your program that performs that function, and see if it is executing when it shouldn't.
- Is a section of code producing an unexpected effect? Make sure you understand the code, especially if it invokes methods in the Java library. Read the documentation for those methods, and try them out with simple test cases. They might not do what you think they do.

To program, you need a mental model of what your code does. If it doesn't do what you expect, the problem might not actually be the program; it might be in your head.

The best way to correct your mental model is to break the program into components (usually the classes and methods) and test them independently. Once you find the discrepancy between your model and reality, you can solve the problem.

Here are some common logic errors to check for:

- Remember that integer division always rounds toward zero. If you want fractions, use `double`. More generally, use integers for countable things and floating-point numbers for measurable things.
- Floating-point numbers are only approximate, so don't rely on them to be perfectly accurate. You should probably never use the `==` operator with `doubles`. Instead of writing `if (d == 1.23)`, do something like `if (Math.abs(d - 1.23) < .000001)`.
- When you apply the equality operator (`==`) to objects, it checks whether they are identical. If you meant to check equivalence, you should use the `equals` method instead.
- By default for user-defined types, `equals` checks identity. If you want a different notion of equivalence, you have to override it.
- Inheritance can lead to subtle logic errors, because you can run inherited code without realizing it. See "Flow of execution" on page ??.

I've got a big, hairy expression and it doesn't do what I expect.

Writing complex expressions is fine as long as they are readable, but they can be hard to debug. It is often a good idea to break a complex expression into a series of assignments to temporary variables:

```
rect.translate((int) Math.round(0.5 * rect.getWidth()),
              (int) Math.round(0.5 * rect.getHeight()));
```

This example can be rewritten as follows:

```
double halfWidth = 0.5 * rect.getWidth();
double halfHeight = 0.5 * rect.getHeight();
int dx = (int) Math.round(halfWidth);
int dy = (int) Math.round(halfHeight);
rect.translate(dx, dy);
```

The second version is easier to read, partly because the variable names provide additional documentation. It's also easier to debug, because you can check the types of the temporary variables and display their values.

Another problem that can occur with big expressions is that the order of operations may not be what you expect. For example, to evaluate $\frac{x}{2\pi}$, you might write this:

```
double y = x / 2 * Math.PI;
```

That is not correct, because multiplication and division have the same precedence, and they are evaluated from left to right. This code computes $\frac{x}{2}\pi$.

If you are not sure of the order of operations, check the documentation, or use parentheses to make it explicit.

```
double y = x / (2 * Math.PI);
```

This version is correct, and more readable for other people who haven't memorized the order of operations.

My method doesn't return what I expect.

If you have a return statement with a complex expression, you don't have a chance to display the value before returning:

```
public Rectangle intersection(Rectangle a, Rectangle b) {
    return new Rectangle(
        Math.min(a.x, b.x), Math.min(a.y, b.y),
        Math.max(a.x + a.width, b.x + b.width)
            - Math.min(a.x, b.x)
        Math.max(a.y + a.height, b.y + b.height)
            - Math.min(a.y, b.y));
}
```

Instead of writing everything in one statement, use temporary variables:

```
public Rectangle intersection(Rectangle a, Rectangle b) {  
    int x1 = Math.min(a.x, b.x);  
    int y1 = Math.min(a.y, b.y);  
    int x2 = Math.max(a.x + a.width, b.x + b.width);  
    int y2 = Math.max(a.y + a.height, b.y + b.height);  
    Rectangle rect = new Rectangle(x1, y1, x2 - x1, y2 - y1);  
    return rect;  
}
```

Now you have the opportunity to display any of the intermediate variables before returning. And by reusing `x1` and `y1`, you made the code smaller too.

My print statement isn't doing anything.

If you use the `println` method, the output is displayed immediately, but if you use `print` (at least in some environments), the output gets stored without being displayed until the next newline. If the program terminates without displaying a newline, you may never see the stored output. If you suspect that this is happening, change some or all of the `print` statements to `println`.

I'm really, really stuck and I need help.

First, get away from the computer for a few minutes. Computers emit waves that affect the brain, causing the following symptoms:

- Frustration and rage.
- Superstitious beliefs (“the computer hates me”) and magical thinking (“the program works only when I wear my hat backward”).
- Sour grapes (“this program is lame anyway”).

If you suffer from any of these symptoms, get up and go for a walk. When you are calm, think about the program. What is it doing? What are possible causes of that behavior? When was the last time you had a working program, and what did you do next?

Sometimes it just takes time to find a bug. People often find bugs when they let their mind wander. Good places to find bugs are buses, showers, and bed.

No, I really need help.

It happens. Even the best programmers get stuck. Sometimes you need another pair of eyes. Before you bring someone else in, make sure you have tried the techniques described in this appendix.

Your program should be as simple as possible, and you should be working on the smallest input that causes the error. You should have print statements in the appropriate places (and the output they produce should be comprehensible). You should understand the problem well enough to describe it concisely.

When you bring someone in to help, give them the information they need:

- What kind of bug is it? Compile-time, run-time, or logic?
- What was the last thing you did before this error occurred? What were the last lines of code that you wrote, or what is the test case that fails?
- If the bug occurs at compile time or run time, what is the error message, and what part of the program does it indicate?
- What have you tried, and what have you learned?

By the time you explain the problem to someone, you might see the answer. This phenomenon is so common that some people recommend a debugging technique called “rubber ducking”. Here’s how it works:

1. Buy a standard-issue rubber duck.
2. When you are really stuck on a problem, put the rubber duck on the desk in front of you and say, “Rubber duck, I am stuck on a problem. Here’s what’s happening...”
3. Explain the problem to the rubber duck.
4. Discover the solution.
5. Thank the rubber duck.

We’re not kidding, it works! See https://en.wikipedia.org/wiki/Rubber_duck_debugging.

I found the bug!

When you find the bug, the way to fix it is usually obvious. But not always. Sometimes what seems to be a bug is really an indication that you don't understand the program, or your algorithm contains an error. In these cases, you might have to rethink the algorithm or adjust your mental model. Take some time away from the computer to think, work through test cases by hand, or draw diagrams to represent the computation.

After you fix the bug, don't just start in making new errors. Take a minute to think about what kind of bug it was, why you made the error, how the error manifested itself, and what you could have done to find it faster. Next time you see something similar, you will be able to find the bug more quickly. Or even better, you will learn to avoid that type of bug for good.

Index

Symbols

() parentheses, 26
; semicolon, 4, 27
= assignment operator, 18
== equals operator, 71, 192
[] square brackets, 108
% remainder operator, 42
<> angle brackets, 227
{ } curly braces, 4, 73

A

abecedarian, 105
abstract class, 274, 276
accessor, 189
accumulator, 114, 122
addition
 integer, 22
 string, 25
 time, 194
address, 33, 46, 191
algorithm, 10, 13
alias, 112, 122
aliasing, 173, 222
allocate, 108, 122
anagram, 125
and operator, 77
angle brackets, 227
args, 152

argument, 54, 59, 65
ArithmeticException, 28, 336
array, 108, 121
 2D, 215
 copying, 111
 element, 109
 index, 109
 length, 112
 of cards, 217
 of objects, 208
 of strings, 203
 printing, 110
ArrayIndexOutOfBoundsException,
 110, 335
ArrayList, 227, 236
Arrays class, 111, 112
assignment, 18, 30, 71
attribute, 168, 180
automatic conversion, 24, 41, 55, 72
AWT, 167, 321, 326

B

base case, 130, 141
BigInteger, 156
binary, 135, 141
binary search, 211, 214
block, 72, 84
Boole, George, 71

- boolean, 71, 80, 84
- bottom-up design, 244, 249
- bounding box, 323, 326
- brackets
 - angle, 227
 - square, 108
- branch, 72, 84
- breakpoint, 306, 310
- bug, 10, 13
- byte code, 6, 13

C

- call stack, 130, 306, 310
- camel case, 52
- Canvas, 321
- Card, 202
- CardCollection, 236
- case-sensitive, 4, 18, 52, 101
- catch, 260
- chaining, 74, 84
- char, 7, 95
- Character, 151, 154
- charAt, 95
- Checkstyle, 305
- class, 4, 12, 196
 - definition, 183
 - relationships, 248
 - utility, 34
 - wrapper, 151
- class diagram, 174, 180, 225, 248
- class variable, 205, 214
- client, 188, 197
- CodingBat, 138
- collection, 227, 230
- Color, 323
- command-line interface, 152, 302, 310, 319
- comment, 12, 84

- documentation, 311, 314
 - end-of-line, 4, 311
 - multiline, 311
- compareTo, 101, 207
- comparison operator, 71
- compile, 5, 13, 330
- compile-time error, 27, 30, 329
- complete ordering, 206
- composition, 59, 65, 248
- computer science, 10, 13
- concatenate, 25, 30, 161
- concrete class, 276
- conditional statement, 72, 84
- constant, 38, 47
- constructor, 185, 197, 203, 217, 222
 - value, 186
- Convert.java, 43, 317
- coordinate, 167, 322, 326
- countdown, 127
- counter, 116
- CPU, 2
- Crazy Eights, 235

D

- De Morgan's laws, 78, 84
- debugger, 306, 310
- debugging, 10, 13, 329
 - by bisection, 332
 - experimental, 11
 - rubber duck, 341
- Deck, 217
- declaration, 17, 30, 167
- decrement, 91, 103
- degrees, 58
- dependent, 189
- description, 315, 320
- design process, 62, 157, 161, 220, 244, 273

deterministic, 115, 122
diagram
 class, 174, 225, 248
 memory, 20, 99, 112, 147, 168,
 186, 193, 204, 217
 stack, 57, 129, 132, 137
divisible, 43
division
 floating-point, 23
 integer, 22, 24
documentation, 44, 174, 312, 320
 Javadoc comments, 314
 Javadoc tags, 315
dot notation, 168, 180
Double, 151
double, 23
doubloon, 105
Doubloon.java, 120
DrJava, 300

E

Echo.java, 35
efficiency, 117, 163, 211, 221, 231,
 237, 295
Eights, 245
element, 108, 109, 121
empty array, 153, 161
empty string, 98, 103
encapsulate, 157, 161
encapsulation, 165, 171
 and generalization, 157
encode, 202, 214
enhanced for loop, 118, 122
equals, 100, 192, 193
equivalent, 192, 197, 206
error
 compile-time, 27, 329
 logic, 29, 329, 338

 message, 11, 27, 330
 rounding, 24
 run-time, 28, 329
 syntax, 331
escape sequence, 9, 13, 95
exception, 28, 329, 335
 Arithmetic, 28
 ArrayIndexOutOfBoundsException, 110
 InputMismatch, 82
 Interrupted, 260
 MissingFormatArgument, 40
 NegativeArraySize, 108
 NullPointerException, 149, 154, 209
 NumberFormatException, 152
 StackOverflow, 130
 StringIndexOutOfBoundsException, 97,
 155
executable, 5, 13
experimental debugging, 11
expression, 22, 30, 59, 79
 big and hairy, 338
extends, 238, 239
extract digits, 43

F

factorial, 130, 141, 162
fibonacci, 134
FileNotFoundException, 336
final, 38, 205, 208
flag, 79, 84
floating-point, 23, 30
flow of execution, 53, 64, 335
for, 92
 enhanced, 118
format specifier, 39, 47, 94
format string, 39, 47, 190
frame, 57, 65

G

garbage collection, 176, 180, 205
generalization, 165, 171, 279, 297
generalize, 158, 161
getter, 189, 197
GitHub, x
Goodbye.java, 7
Google style, 8
Graphics, 256, 321
Greenfield, Larry, 11
GuessStarter.java, 49

H

hacker, 81, 85, 154
Hand, 240
hanging, 333
hardware, 2, 12
HAS-A, 248, 249, 256, 275
Hello.java, 3
helper method, 219, 230
hexadecimal, 33, 40, 191
high-level language, 5, 12
histogram, 116, 122, 215
HTML, 174, 311, 316

I

IDE, 299, 309
identical, 192, 197
if statement, 72
ImageIO, 292
immutable, 150, 161, 177, 208
import statement, 35, 47
increment, 91, 102
incremental development, 62, 65
independent, 189
index, 103, 109, 121, 209
indexOf, 98
infinite loop, 90, 102, 333

infinite recursion, 186, 334
information hiding, 184, 197
inheritance, 238, 248, 249, 279, 290
initialize, 19, 30, 79
inner loop, 94
InputMismatchException, 82
instance, 183, 196
instance method, 191, 195, 197
instance variable, 184, 196
instantiate, 183, 196
Integer, 151
integer division, 22, 24
interactions, 301
interface, 291
interpret, 5, 13
InterruptedException, 260
invoke, 51, 64
IS-A, 248, 249, 256, 275, 287
iteration, 92, 97, 103
iterative, 127, 140

J

JAR, 300, 309
Java Tutor, 57, 140
java.awt, 167, 321
java.io, 33
java.lang, 35
java.math, 156
java.util, 34
javac, 6, 154, 302
Javadoc, 174, 300, 311, 316, 320
javax.imageio, 292
JDK, 300, 309, 311
JFrame, 259, 322
JVM, 6, 300, 309

K

keyword, 18, 30, 185

L

- language
 - elements, 36
 - high-level, 5
 - low-level, 5
- leap of faith, 133, 141, 224
- length
 - array, 112
 - string, 97
- library, 34, 46, 173
- Linux, 11
- literal, 38, 47
- local variable, 55, 65
- Logarithm.java, 83
- logic error, 29, 31, 329, 338
- logical operator, 77, 84, 206
- long, 58
- loop, 90, 102
 - for, 92
 - infinite, 90
 - nested, 94, 209
 - search, 210
 - while, 89
- loop body, 90, 102
- loop variable, 94, 103, 110
- low-level language, 5, 12

M

- magic number, 38, 47
- main, 4, 52
- map to, 202
- Math, 58
- memory, 2, 12
- memory diagram, 20, 30, 99, 112, 147, 168, 186, 193, 204, 217
- mental model, 57, 338
- merge, 223
- merge sort, 221, 230

- method, 4, 12
 - abstract, 274
 - accessor, 189
 - boolean, 80
 - constructor, 185
 - equals, 192, 193
 - getter, 189
 - helper, 219
 - instance, 191, 195
 - modifier, 238
 - mutator, 190
 - parameters, 55
 - setter, 190
 - static, 194
 - toString, 191
 - value, 60
- Mickey Mouse, 324
- MissingFormatArgumentException, 40
- modifier method, 238
- modulo, 42, 47
- modulus, 43, 47
- multidimensional array, 254, 265
- mutable, 171, 177, 180
- mutator, 190

N

- NaN, 85
- NegativeArraySizeException, 108
- neighbor, 251, 261
- nested, 336
 - arrays, 256
 - conditions, 75
 - loops, 94
- nesting, 74, 84, 209
- new, 35, 49, 108, 167, 185
- newline, 7, 13, 128
- NewLine.java, 52

next
 Scanner, 82
nextInt
 Random, 115
 Scanner, 37
nondeterministic, 115, 122
not operator, 77
null, 149, 208
NullPointerException, 149, 154,
 335
NumberFormatException, 152

O

object, 147, 161
 array of, 208
 as parameter, 169
 displaying, 191
 mutable, 171
 type, 183
Object class, 239
object code, 5, 13
object-oriented, 147, 160, 172, 248,
 276
off-by-one, 222, 230
operand, 22, 30
operator, 22, 30
 assignment, 71
 cast, 42
 logical, 77, 206
 new, 35, 49, 108, 167, 185
 redirection, 304
 relational, 71
 remainder, 42
 string, 25
or operator, 77
order of operations, 26, 30, 339
ordering, 206
outer loop, 94

overload, 186, 222
overloaded, 99, 103
override, 191, 197, 249

P

package, 33, 46
paint, 322
palindrome, 145
param tag, 315
parameter, 54, 65, 169
 multiple, 55
parameter passing, 54, 65
parentheses, 26
parse, 28, 31, 152, 161
partial ordering, 206
pi, 58
Pile, 227
pixel, 323, 326
Player, 242
Point, 167
polymorphism, 287, 291, 297
portable, 5, 13
precedence, 26, 339
primitive, 147, 161
print, 7, 191
print statement, 3, 12, 190, 336, 340
printArray, 111
printDeck, 209, 218
printf, 39, 101, 190
println, 4
printPoint, 169
printTime, 190
PrintTime.java, 56
PrintTwice.java, 54
private, 174, 184, 188
problem solving, 1, 12
processor, 2, 12
program, 2, 12

programming, 3, 12
prompt, 37, 47
protected, 274
pseudocode, 219, 230
pseudorandom, 115, 122
public, 51

Q

quote mark, 7, 95, 331

R

radians, 58
RAM, 2
Random, 115
randomInt, 220
rank, 201
rational number, 198
Rectangle, 170
recursion, 130
 infinite, 186, 334
recursive, 127, 140
redirection operator, 304, 310
reduce, 114, 116, 122
refactor, 273, 276
reference, 109, 122, 167, 172, 204,
 222
relational operator, 71, 84
remainder, 42
replace, 150
repository, x
return, 61, 83, 170
 inside loop, 211
return statement, 339
return tag, 315
return type, 60, 65
return value, 60, 65
RGB, 324, 326
rounding error, 24, 30

row-major order, 255, 265
rubber duck, 341
run-time error, 28, 31, 82, 209, 329

S

scaffolding, 63, 65
Scanner, 34
scope, 57, 65, 175
Scrabble, 105, 197
search, 114, 122
selection sort, 220, 230
semicolon, 4, 27
sequential search, 210, 214
setter, 190, 197
shadowing, 188, 197
short circuit, 78, 84, 155
shuffle, 219
signature, 313, 320
sleep, 260
sort
 merge, 221
 selection, 220
source code, 5, 13, 173
specialization, 279, 297
sprite, 292, 297
src.zip, 173
stable configuration, 252
stack diagram, 57, 65, 129, 132, 137
stack trace, 41, 47, 335
StackOverflowError, 130, 186, 333,
 336
state, 20, 30, 45, 129, 242, 253, 270
statement, 3, 12
 assignment, 18
 catch, 260
 conditional, 72
 declaration, 17, 167
 else, 72

- for, 92
- if, 72
- import, 35
- initialization, 79
- print, 7, 191, 336, 340
- return, 61, 83, 170, 211, 339
- switch, 75
- throw, 285
- try, 260
- while, 89
- static, 185, 194, 205
- static context, 226, 230
- string, 7, 13
 - array of, 203
 - comparing, 100, 148
 - format, 102, 192
 - length, 97
 - operator, 25
 - reference to, 204
- String class, 167
- StringBuilder, 179, 231
- StringIndexOutOfBoundsException,
 - 97, 155
- stub, 62, 65
- style guide, 8
- subclass, 238, 249
- subdeck, 222
- substring, 99
- suit, 201
- superclass, 239, 249
- Surprise.java, 177
- swapCards, 220
- switch statement, 75
- syntax, 329
- syntax errors, 331
- System.err, 82, 309
- System.in, 34, 147, 304
- System.out, 33, 147, 304

T

- table
 - two-dimensional, 157
- tag, 315, 320
- temporary variable, 61, 65, 339
- terminal, 302
- testing, 63, 82, 232, 303
- text editor, 300, 309
- this, 176, 185, 226, 236
- Thread.sleep, 260
- throw, 285
- Time, 184
 - addition, 194
- toCharArray, 120
- token, 36, 47
- toLowerCase, 150
- top-down design, 220, 230, 244
- Torvalds, Linus, 11
- toString, 169, 191
- toUpperCase, 150
- tracing, 57, 213, 306, 335
- traversal, 113, 122, 210, 256
- try, 260
- type, 30
 - array, 108
 - boolean, 71, 79
 - char, 17, 95
 - double, 23
 - int, 17
 - long, 25, 58
 - object, 183
 - String, 7, 17, 167
 - void, 51
- type cast, 42, 47

U

- UML, 174, 180, 225, 248
- Unicode, 96, 103

unit test, 307, 310
utility class, 34, 111

V

validate, 81, 85, 154
value, 17, 30
value constructor, 186
value method, 60
variable, 17, 29

- instance, 184
- local, 55
- loop, 94, 110

- private, 174, 184, 188
- static, 205
- temporary, 61, 339

virtual machine, 6, 13, 300
void, 51, 60, 64

W

War (card game), 227
while, 89
wildcard, 305, 310
wrapper class, 151, 161
wrapper method, 231

About the Authors

Allen Downey is a professor of computer science at Olin College of Engineering. He has taught computer science at Wellesley College, Colby College, and UC Berkeley. He has a PhD in computer science from UC Berkeley and master's and bachelor's degrees from MIT. Allen is the creator of the best-selling Think series for O'Reilly, which includes *Think Python*, *Think Complexity*, *Think DSP*, and *Think Bayes*.

Chris Mayfield is an associate professor of computer science at James Madison University, with a research focus on CS education and professional development. He has a PhD in computer science from Purdue University and bachelor's degrees in CS and German from the University of Utah.